

# 머신러닝 교재 3

## 머신러닝 분류 및 군집화 알고리즘 정리

---

기계학습에서는 데이터의 특성과 목적에 따라 다양한 알고리즘이 사용된다.

그중 **k-최근접 이웃 (KNN)**, **서포트 벡터 머신 (SVM)**, **결정 트리 (Decision Tree)**은 분류 그리고 **k-평균 클러스터링 (k-Means)**은 군집화 문제에서 자주 사용되는 대표적인 알고리즘이다.

### 1. kNN (k-Nearest Neighbors)

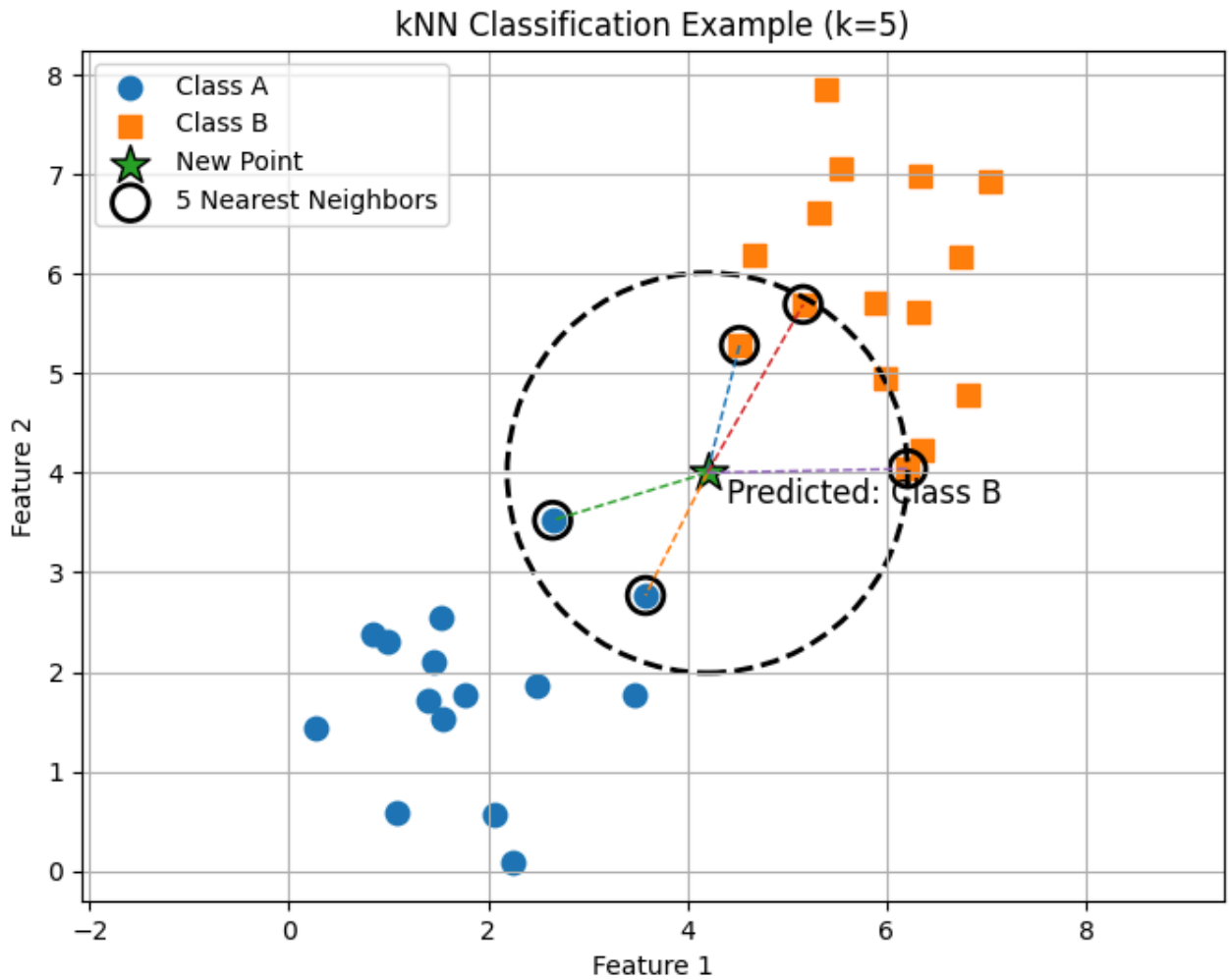
kNN은 학습한 후 저장된 데이터 세트와 하나의 데이터 요소를 근접성을 비교하고 예측을 수행하는 머신러닝 알고리즘이다.

분류 알고리즘으로서 kNN은 이웃한 데이터 중 다수 세트에 새로운 데이터 요소를 할당한다. 회귀 알고리즘인 kNN은 쿼리 지점에 가장 가까운 값의 평균을 기반으로 예측을 수행한다.

kNN은 **지도 학습 알고리즘**으로, durltj 'k'는 분류 또는 회귀 문제에서 최근접 이웃의 수를 나타내고, 'NN'은 k로 선택된 최근접 이웃을 나타낸다.

#### 작동 원리

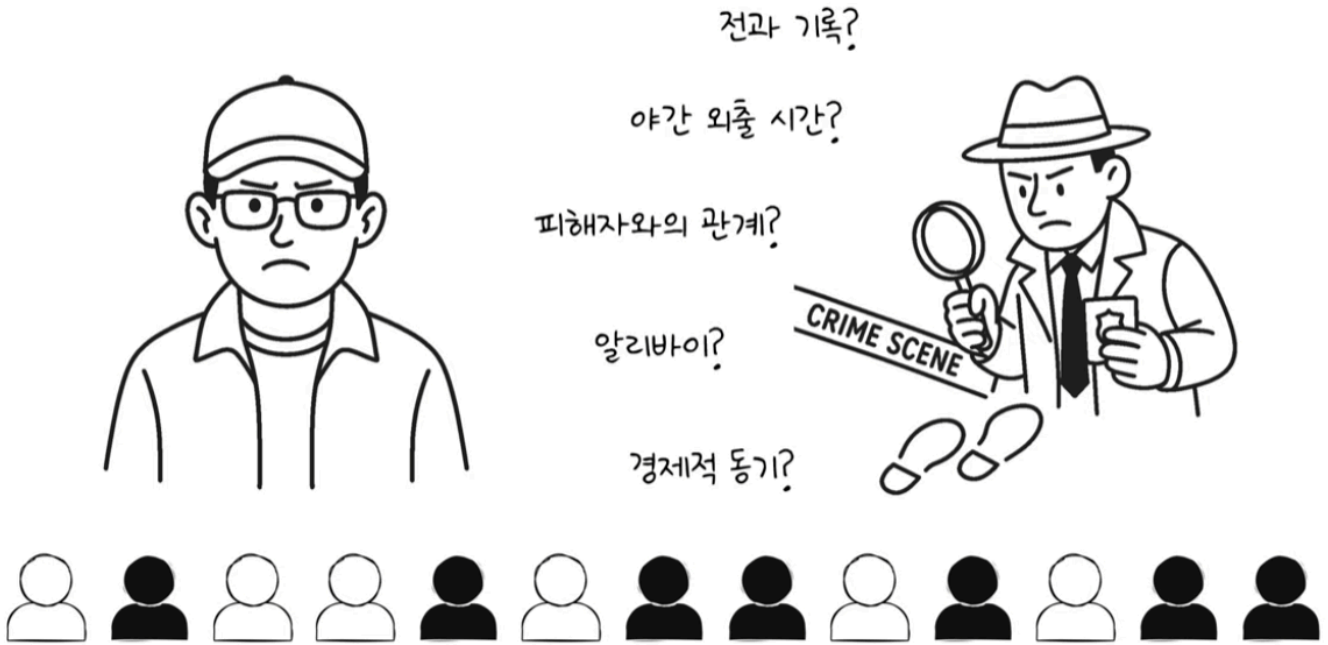
- 새로운 입력과 훈련 데이터 간의 거리를 계산 (보통 유클리디안 거리)한다. 거기서 가장 가까운 k개의 이웃을 선택한다.
- 이웃 중 가장 많이 나타나는 클래스(다수결)로 분류



## 예시

만약 우리를 범죄 수사관이라고 가정해 본다. 용의자를 선별하고 이 사람이 진짜 범죄자인지 확인하기 위해, 사람들의 행동 패턴을 분석한다.

그래서 이 용의자와 행동이 가장 비슷한 K명을 찾아서 기존의 범죄자들과 용의자의 행동 패턴이 얼마나 유사한지 확률적으로 비교해 볼 수 있다.



예를 들어 아래 표와 같은 사건 데이터가 있을 때 두 가지 특징을 고려한다. 해당 특징을 기반으로 프로파일링을 시도한다.

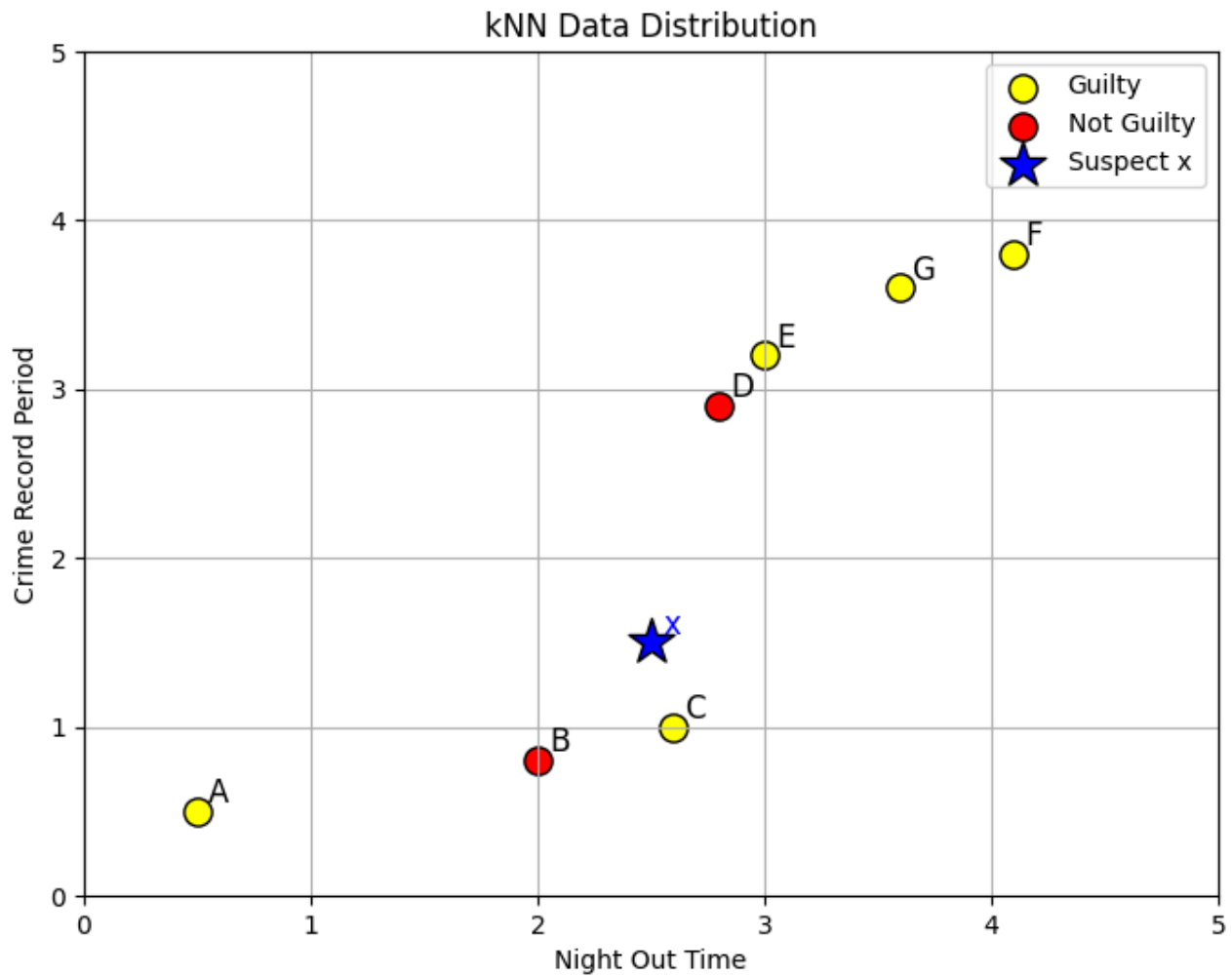
| 사람      | 야간 외출시간 | 범죄 복역기간 | 혐의점 |
|---------|---------|---------|-----|
| A       | 0.5     | 0.5     | 혐의  |
| B       | 2.0     | 0.8     | 무혐의 |
| C       | 2.6     | 1.0     | 혐의  |
| D       | 2.8     | 2.9     | 무혐의 |
| E       | 3.0     | 3.2     | 혐의  |
| F       | 4.1     | 3.8     | 혐의  |
| G       | 3.6     | 3.6     | 혐의  |
| x (용의자) | 2.5     | 1.5     |     |

우선 다음과 같이 용의자 X와 다른 사람들간의 유사도를 조사한다.

| 데이터 | 분류  | 거리 계산식                                 | 거리   |
|-----|-----|----------------------------------------|------|
| A   | 혐의  | $\sqrt{(2.5 - 0.5)^2 + (1.5 - 0.5)^2}$ | 2.24 |
| B   | 무혐의 | $\sqrt{(2.5 - 2.0)^2 + (1.5 - 0.8)^2}$ | 0.86 |
| C   | 혐의  | $\sqrt{(2.5 - 2.6)^2 + (1.5 - 1.0)^2}$ | 0.51 |
| D   | 무혐의 | $\sqrt{(2.5 - 2.8)^2 + (1.5 - 2.9)^2}$ | 1.43 |
| E   | 혐의  | $\sqrt{(2.5 - 3.0)^2 + (1.5 - 3.2)^2}$ | 1.77 |

| 데이터 | 분류 | 거리 계산식                                 | 거리   |
|-----|----|----------------------------------------|------|
| F   | 혐의 | $\sqrt{(2.5 - 4.1)^2 + (1.5 - 3.8)^2}$ | 2.80 |
| G   | 혐의 | $\sqrt{(2.5 - 3.6)^2 + (1.5 - 3.6)^2}$ | 2.37 |

해당 표를 그래프로 표현한다.

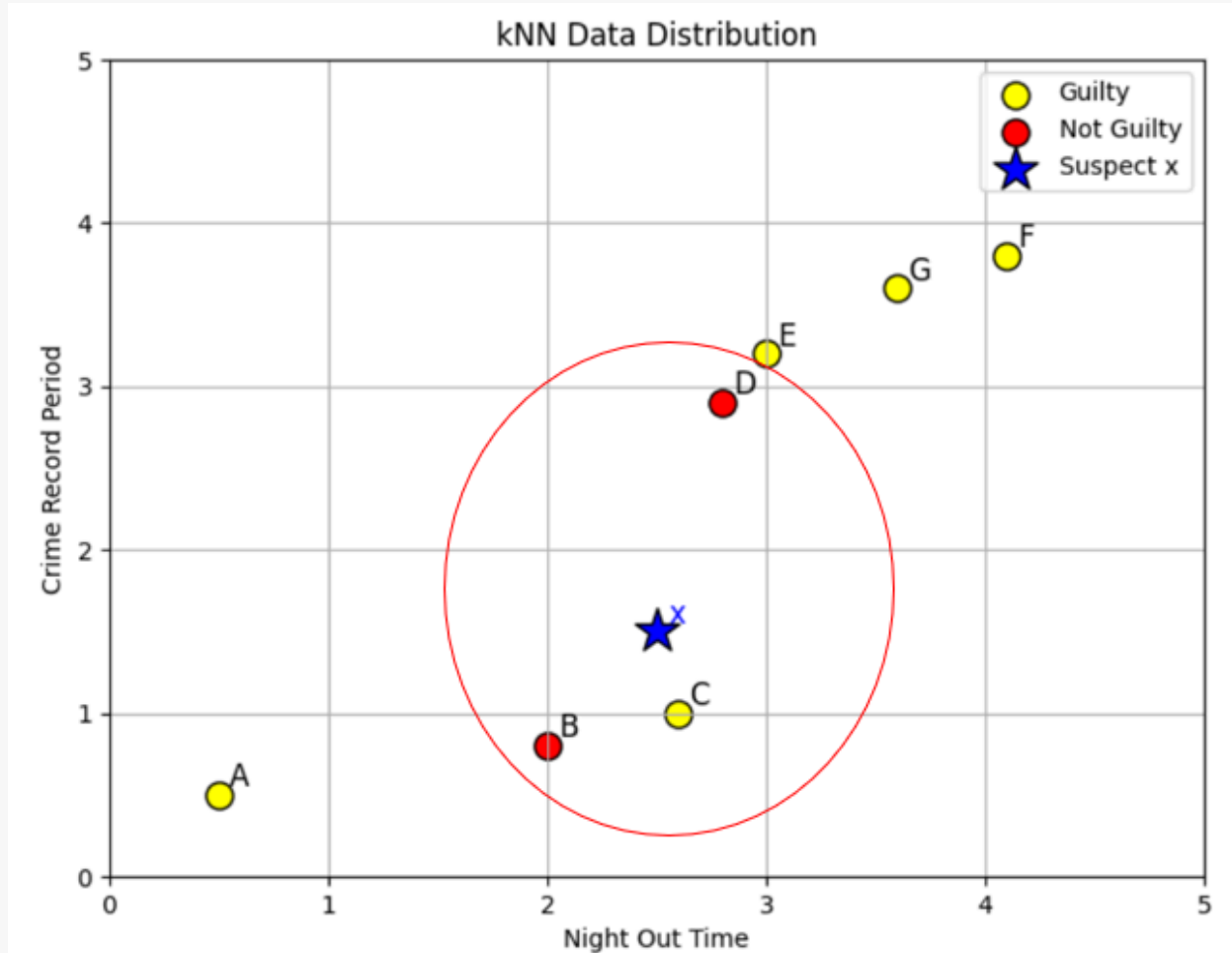


X와 유사도가 가까운 순으로 정렬하면 아래 표와 같다

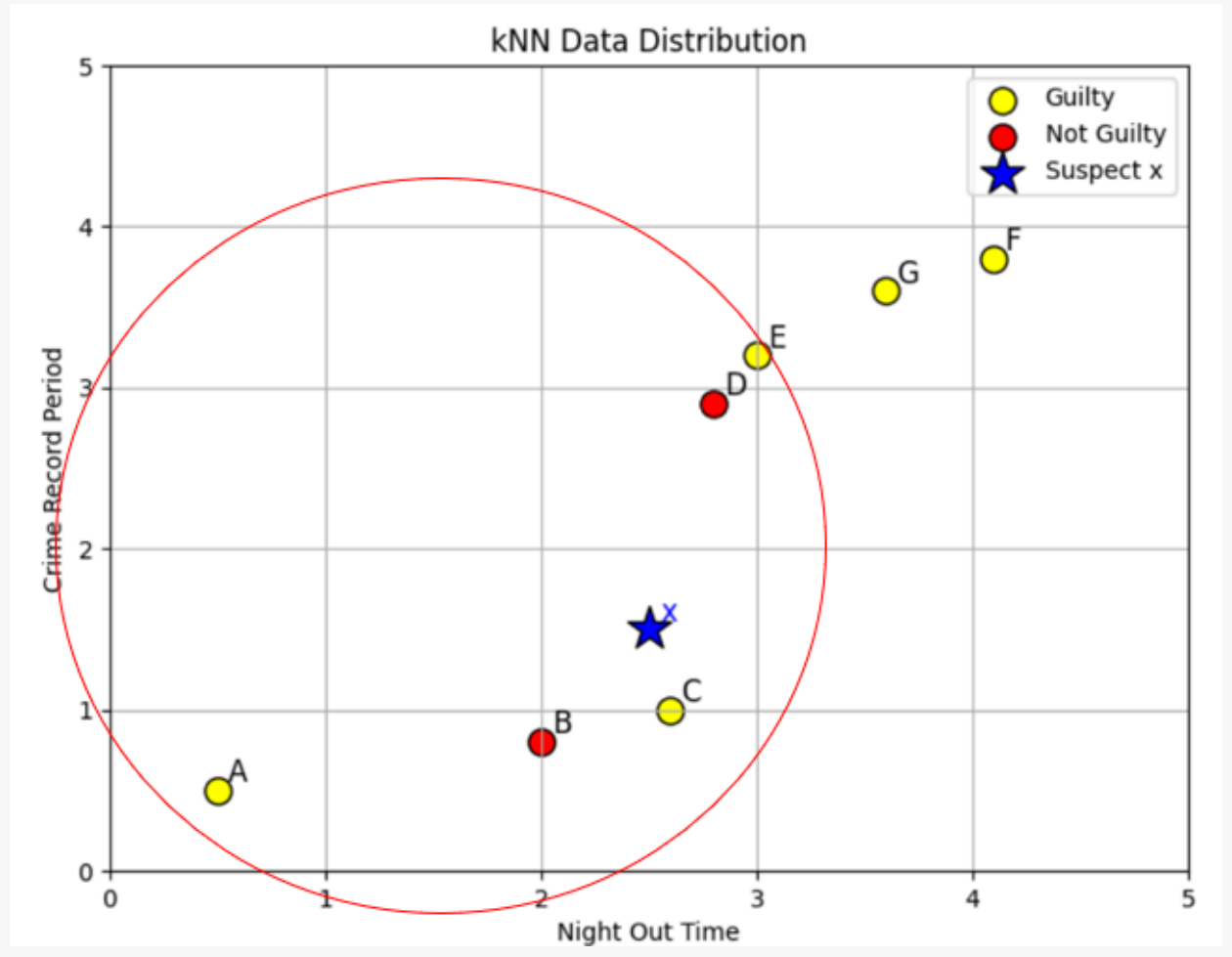
| 순위 | 데이터 | 분류  | 거리   |
|----|-----|-----|------|
| 1  | C   | 혐의  | 0.51 |
| 2  | B   | 무혐의 | 0.86 |
| 3  | D   | 무혐의 | 1.43 |
| 4  | E   | 혐의  | 1.77 |
| 5  | A   | 혐의  | 2.24 |
| 6  | G   | 혐의  | 2.37 |
| 7  | F   | 혐의  | 2.80 |

여기서 K 가 3이라는 말은 용의자 X와 가장 가까운 세 명의 이웃을 정한다는 뜻이다. 그래서 k=3일때, 유사한 특성을 지닌 세 사람중, 두 명이 무혐의이기 때문에, 이 용의자는 무혐의라고 판단한다. 하자미나 K=1로 잡을 때 혹은 K=5로 잡을 땐 혐의 가 더 많아(K=1: 혐의 1, K=5: 혐의 3, 무혐의 2) 혐의가 높다고 판단한다.

K=3일때



K=5



이처럼 K가 너무 작으면 노이즈에 너무 민감해 지고 K 값이 너무 크면 전체 평균만 보게 되어 국소 정보가 사라질 수 있다. kNN은 **K 값에 따라 결과가 흔들리기 때문에**, 겉보기엔 다소 불안정한 모델처럼 보일 수 있다. 하지만 판단의 근거가 되는 좋은 특징들로 데이터가 풍부해진다면, kNN은 단순하면서도 정확한 분류 모델이 될 수 있다.

#### 장점

- 구현이 간단하고 직관적이다.
- 훈련 과정이 따로 없고, 저장만 해두면 된다.
- 다양한 클래스와 복잡한 결정 경계도 잘 표현할 수 있다.

#### 단점

- 모든 훈련 데이터와 거리를 계산하므로 예측 시 계산량이 크다.
- 고차원 데이터에서는 성능이 저하됨(차원의 저주).
- 이상치에 민감하고, 데이터가 불균형할 경우 성능이 낮아질 수 있다.

이상치: 데이터들 사이에서 다른 값들과 너무 멀리 떨어져 있는 데이터를 말한다. 예를들어 대부분의 학생 점수가 60~80점인데, 한 명만 5점이거나 100점이면 그 값은 이상치일 가능성이 있다.

## 거리 측정 수식

1. 유클리드 거리 (Euclidean Distance): 가장 일반적인 직선 거리 계산법으로, 점과 점 사이의 거리를 구할 때 사용한다.

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

2. 맨해튼 거리 (Manhattan Distance): 격자판을 따라 이동하는 거리와 같으며, 도시 블록 거리라고도 한다.

$$d(x, y) = \sum_{i=1}^n |x_i - y_i|$$

3. 민코프스키 거리 (Minkowski Distance): 유클리드 거리와 맨해튼 거리를 일반화한 수식입니다.  $p$ 값에 따라 거리 계산 방식이 달라집니다.

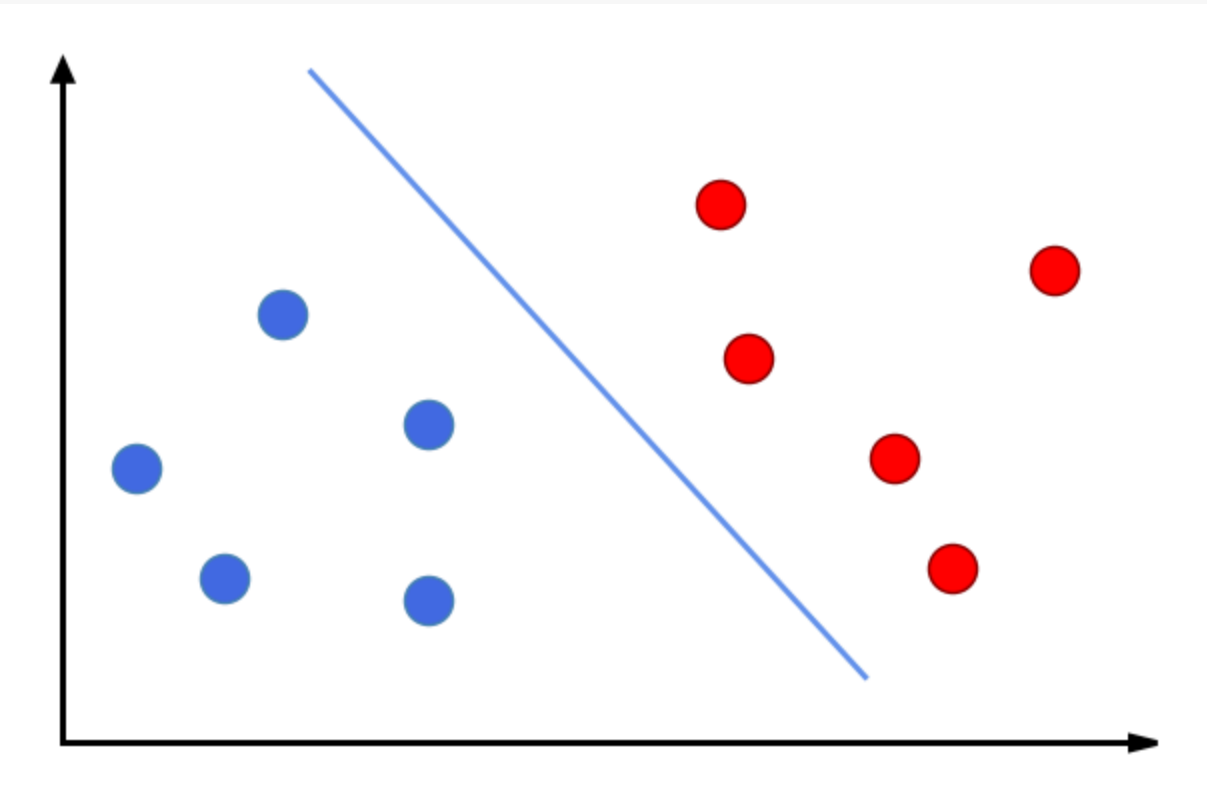
$$d(x, y) = \left( \sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}}$$

- 해당 수식들은 당장 자세하게 이해할 필요는 없다.

## SVM (Support Vector Machine)

SVM은 데이터를 분류할 때 두 집단을 가장 잘 나누는 경계선을 찾는 모델이다. SVM의 목표 클래스 간의 경계를 최대한 넓게 확보하는 결정 경계를 찾아 분류하는 지도 학습 알고리즘이다.

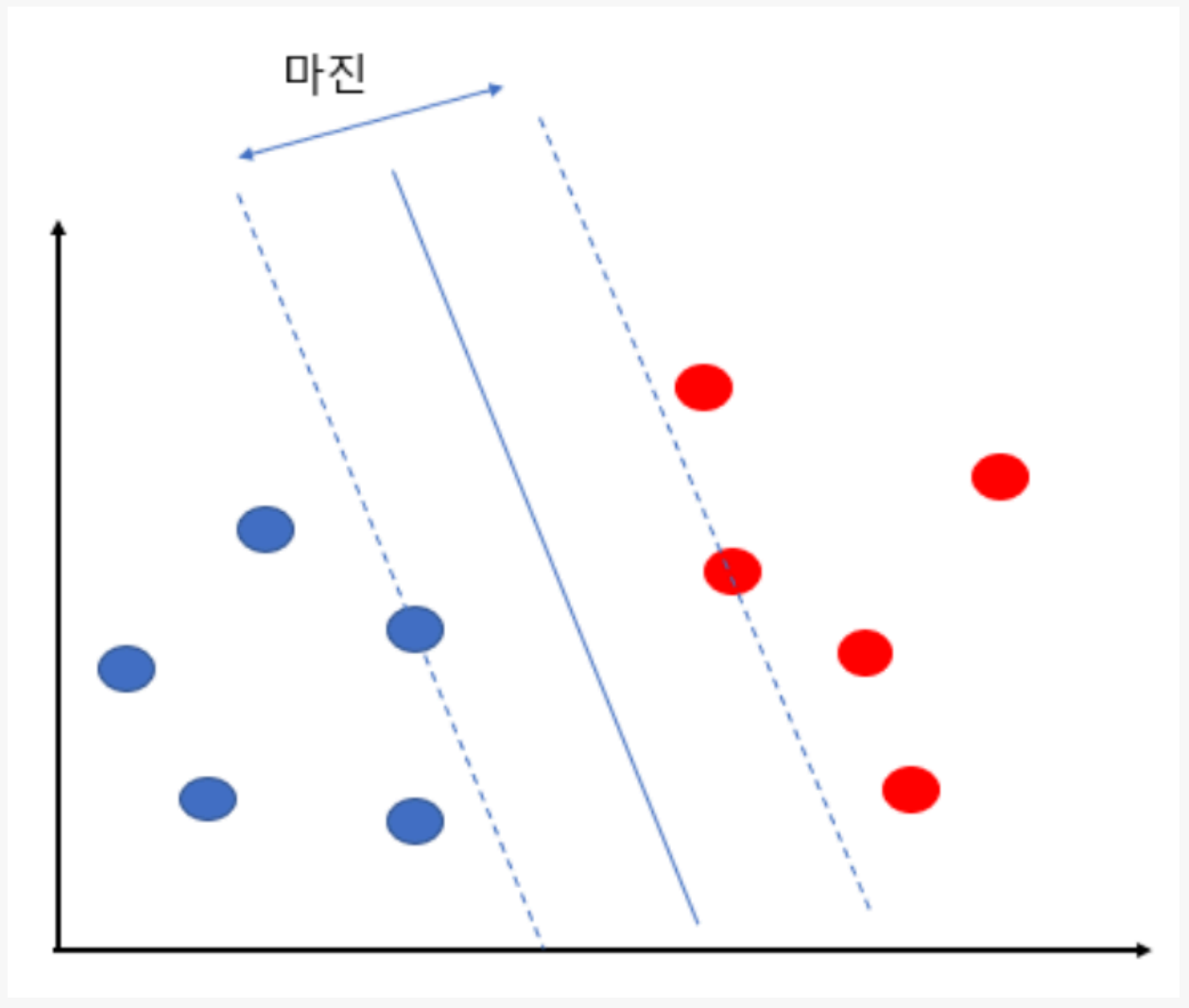
SVM 그래프



위 그래프에는 다음과 같이 클래스 0, 1로 구분되는 10개의 데이터가 있다. 클래스 빨간색과 파란색을 정확히 분류하는 것을 목표로한다. SVM은 클래스를 분류할 수 있는 **최적의 경계**를 찾고자 한다.



마진 설명 그래프

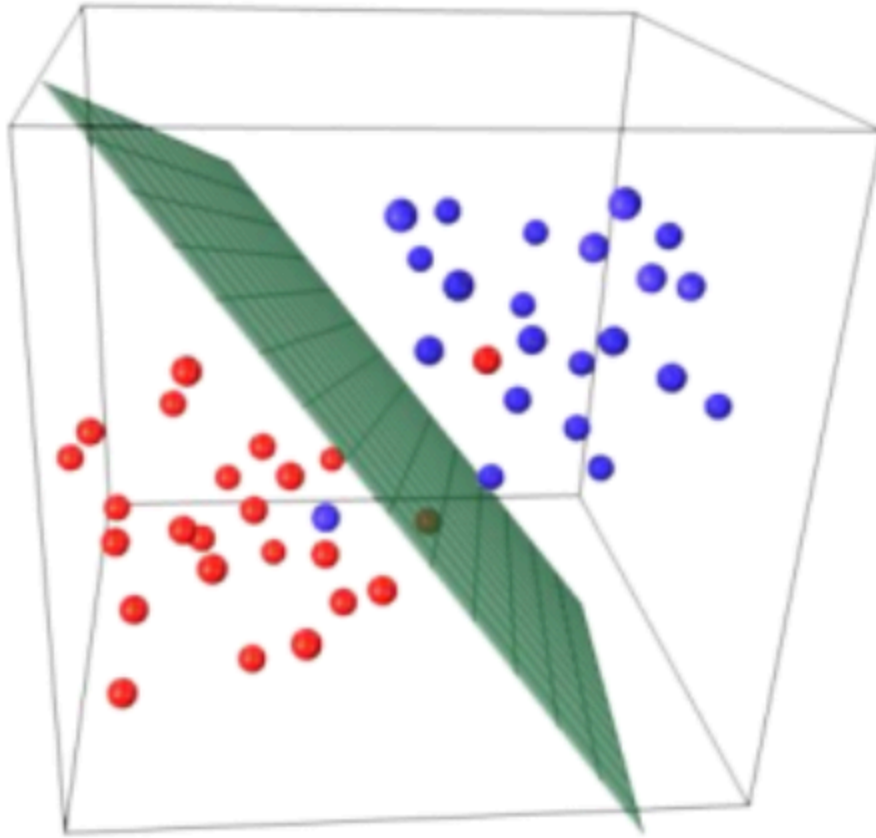


SVM에서 말하는 **최적의 경계**는 각 클래스의 말단에 위치한 데이터들 사이의 거리를 의미한다. 이를 **마진**이라한다. SVM은 분류를 할 때 이 마진을 최대화시키고자 한다. 즉 마진을 최대화 할 수 있는 경계를 찾는것이다.

**support vector(서포트 벡터)**는 마진에서 가장 가까이 위치해 있는 각 클래스 데이터이다. 위 그림에 서는 점선 위에 있는 두 개의 데이터가 support vector이다.

- 이를 **서포트 벡터**라고 하는 이유는 이 데이터들의 위치에 따라서 경계(초평면)의 위치가 달라지기 때문에 초평면 함수를 지지한다는 의미에서 명명되었다.

SVM에서 N차원 공간을 (N-1)차원으로 나눌 수 있는 초평면을 찾는다는 것은 2차원이 아닌 고차원에서 도 분류할 수 있는 경계를 말한다.



위 그림을 보면 데이터가 3차원에 존재한다. (배열이 3차원 형태로 되어있다는 것을 의미한다.) 데이터가 3차원이라면 2차원 공간으로 나눈다는 것을 일반화하여  $N$ ,  $(N-1)$ 차원으로 표현한다는 것이다.

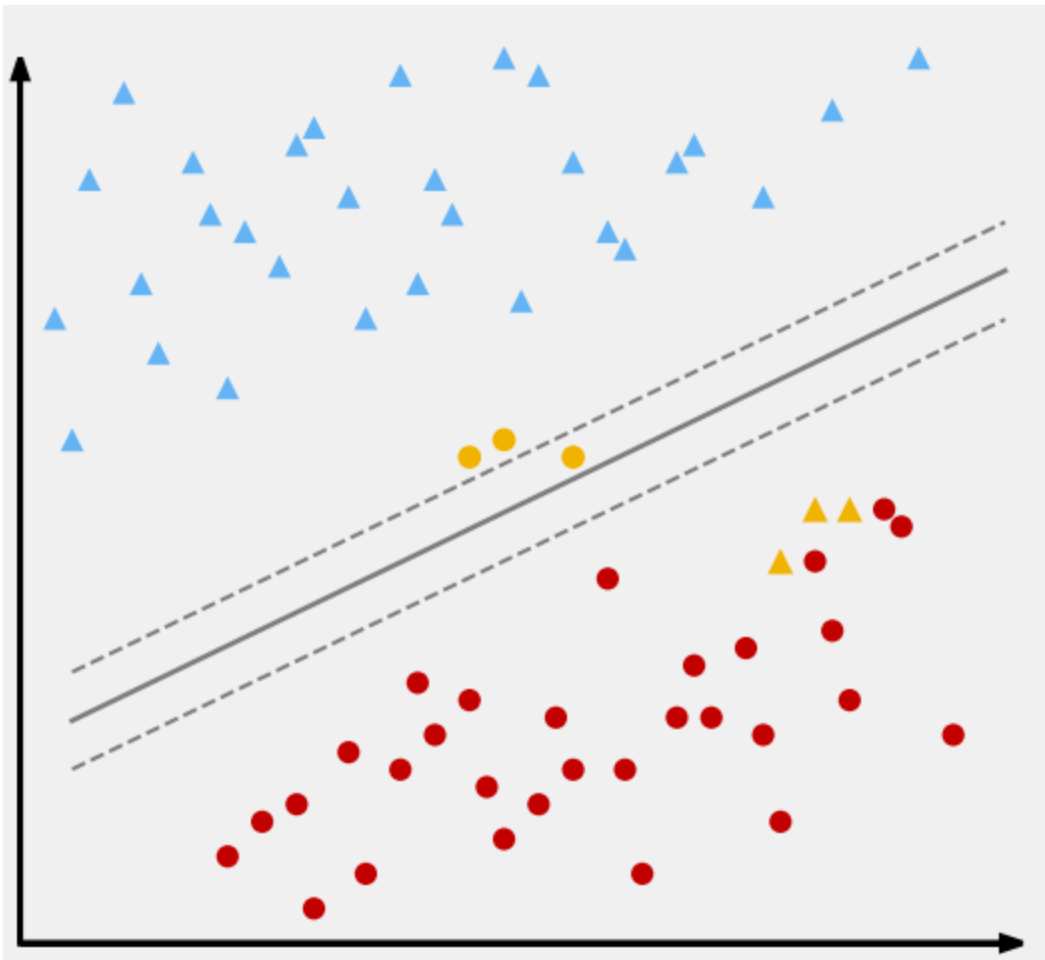
정리하면 SVM은 데이터를 분리하는 분리하는 최적의 초평면, 즉 최대 마진이 되도록 클래스를 분류하는 기법이다.

- 작동 원리
  1. 두 클래스 사이에서 마진(여백)이 최대가 되는 선형 결정 경계(초평면)를 탐색한다.
  2. 결정 경계에 가장 가까운 점들을 서포트 벡터 라고 하며, 이를 경계로 결정한다.
  3. 선형 분리가 어려운 경우, 커널 트릭을 사용해 고차원으로 변형하여 분류 수행한다.

## SVM 개념과 종류

- SVM은 선형 SVM과 비선형 SVM으로 나눌 수 있다.
- 모든 경우에 데이터를 선형으로 분류할 수는 없기에 나눠서 이해할 필요가 있다.

### 선형 SVM

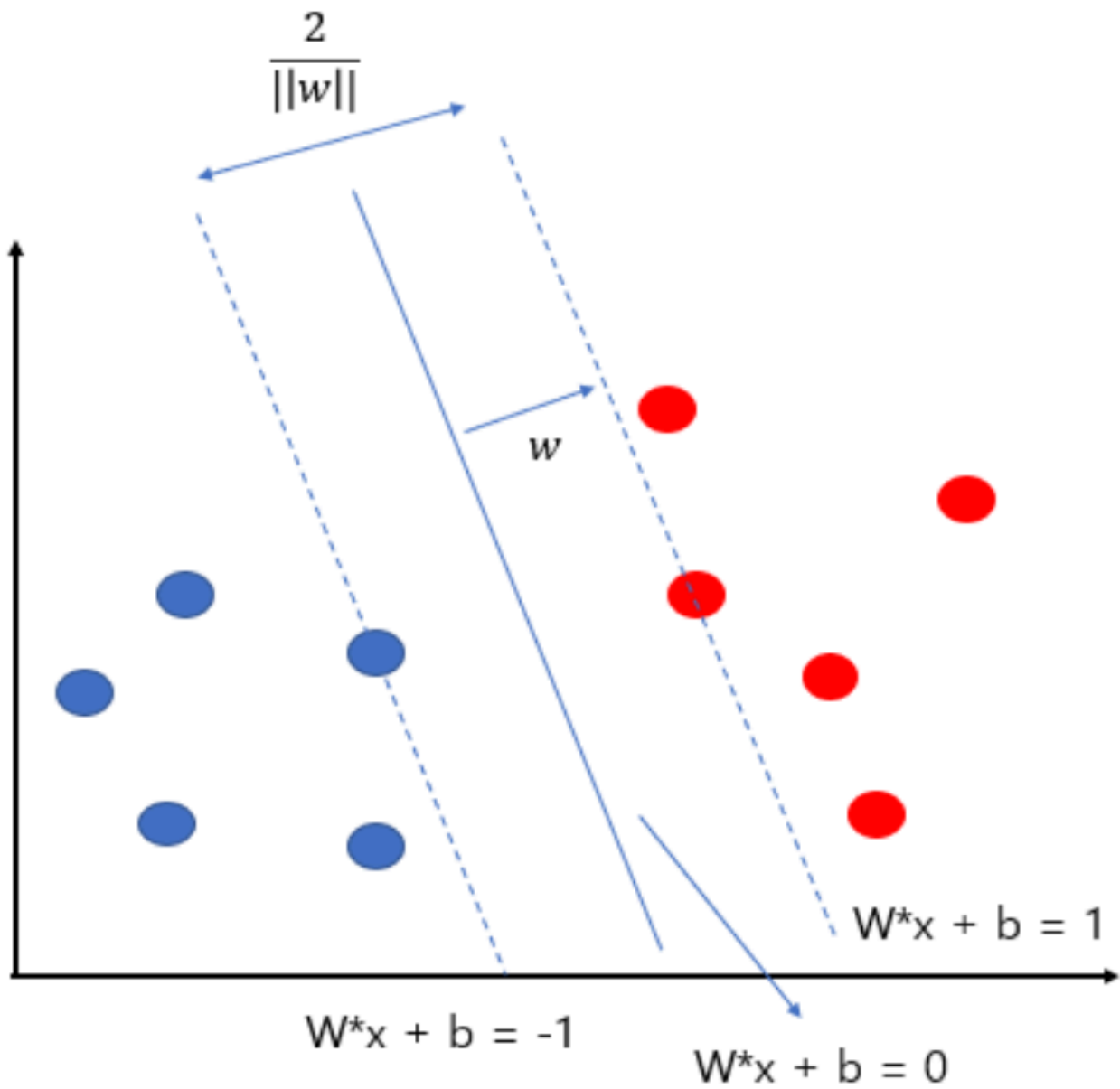


A) Linear Separation

선형 SVM은 선형으로 데이터의 클래스를 나눌 수 있는 경우 사용하게 되며 크게 하드마진과 소프트 마진 방식으로 나눈다.

### 1. 하드 마진

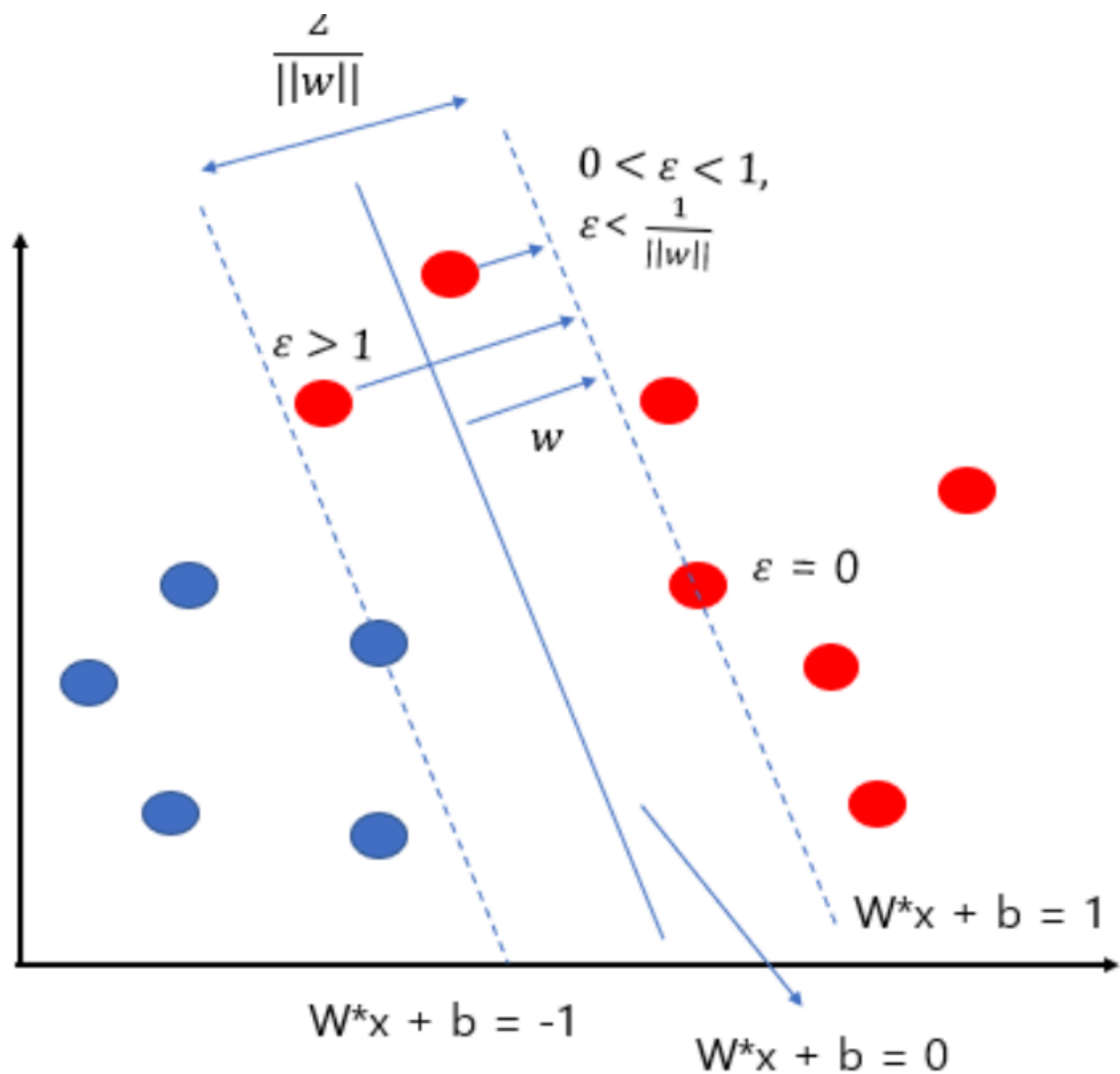
**하드 마진**은 두 클래스를 분류할수 있는 최대 마진의 초평면을 찾는 방법이다. 단, 모든 훈련데이터는 마진의 바깥쪽에 위치하게 선형으로 구분해야 한다. *다시 말해 하나의 오차도 허용하면 안된다는 것을 의미한다.*



하지만 모든 데이터를 선형으로 오차없이 나눌 수 있는 결정 경계를 찾는 것은 사실상 어렵다. 그래서 이를 해결하고자 나온 개념이 소프트 마진이다.

## 2. 소프트마진

**소프트 마진**은 완벽하게 분류하는 초 평면을 찾는 것이 아니라 어느 정도의 오분류를 허용하는 방식이다.



정리하자면 완벽한 분류가 힘들기 때문에 소프트 마진을 활용하는데 특정 크기만큼 초평면 위, 아래로 오차를 허용하는 것이다.

다만 허용하는 범위가 작으면 마진이 줄어들어 오류는 적지만 과적합의 위험이 생기며, 허용하는 범위가 커지면 마진이 크고 오류가 증가하기 쉽다.

### 비선형 분리



선형 분리가 불가능한 상태이지만 이를 해결하기 위해 데이터를 2차원으로 변환한다. 2차원 상태에서는 선형 분리가 가능한 형태가 되게 만든다.

**비선형분리의 기본개념**은 선형 분리가 불가능한 입력공간을 선형분리가 가능한 고차원 특성공간으로 보내 선형분리를 진행한다. 그 후 기존의 입력 공간으로 변환하면 비선형 분리를 하게 된다.

다만 단점이 있는데 차원의 개수가 커지면 계산량이 증가한다는 것이다. 이는 다른 머신러닝에 기법, 딥러닝 기법에서도 마찬가지로 차원이 커지면 거리가 증가하고, 특성이 증가하며 처리량이 커지게 된다. 이를 해결하기 위해 데이터 수를 줄이거나 특성수를 줄이기도 한다.

- 장점
  - 마진을 최대함으로써 일반화 성능이 우수.
  - 다양한 커널 함수로 비선형 문제도 처리 가능
  - 과적합 위험이 적다.
- 단점
  - 훈련 시간이 오래 걸릴 수 있으며, 대규모 데이터셋에 비효율적
  - 커널과 하이퍼파라미터 선택에 민감
  - 예측 확률 값을 직접 제공하지 않음

```

from sklearn.datasets import make_blobs
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np

# 1. Create a simple 2-class dataset
X, y = make_blobs(n_samples=100, centers=2, random_state=6, cluster_std=1.2)

# 2. Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Train SVM with linear kernel
svm_clf = SVC(kernel='linear', C=1.0)
svm_clf.fit(X_scaled, y)

# 4. Visualization
def plot_svm_decision_boundary(clf, X, y):
    plt.figure(figsize=(8, 6))
    plt.scatter(X[:, 0], X[:, 1], c=y, s=50, cmap='bwr', edgecolors='k')

    ax = plt.gca()
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()

    # Create grid to evaluate model
    xx = np.linspace(xlim[0], xlim[1], 30)
    yy = np.linspace(ylim[0], ylim[1], 30)
    YY, XX = np.meshgrid(yy, xx)
    xy = np.vstack([XX.ravel(), YY.ravel()]).T
    Z = clf.decision_function(xy).reshape(XX.shape)

    # Plot decision boundary and margins
    plt.contour(
        XX, YY, Z, colors='k',
        levels=[-1, 0, 1], alpha=0.7,
        linestyles=['--', '-', '--']
    )

    # Plot support vectors
    plt.scatter(
        clf.support_vectors_[:, 0], clf.support_vectors_[:, 1],
        s=100, linewidth=1, facecolors='none', edgecolors='k',
        label='Support Vectors'
    )

    plt.title("SVM Decision Boundary with Margins")

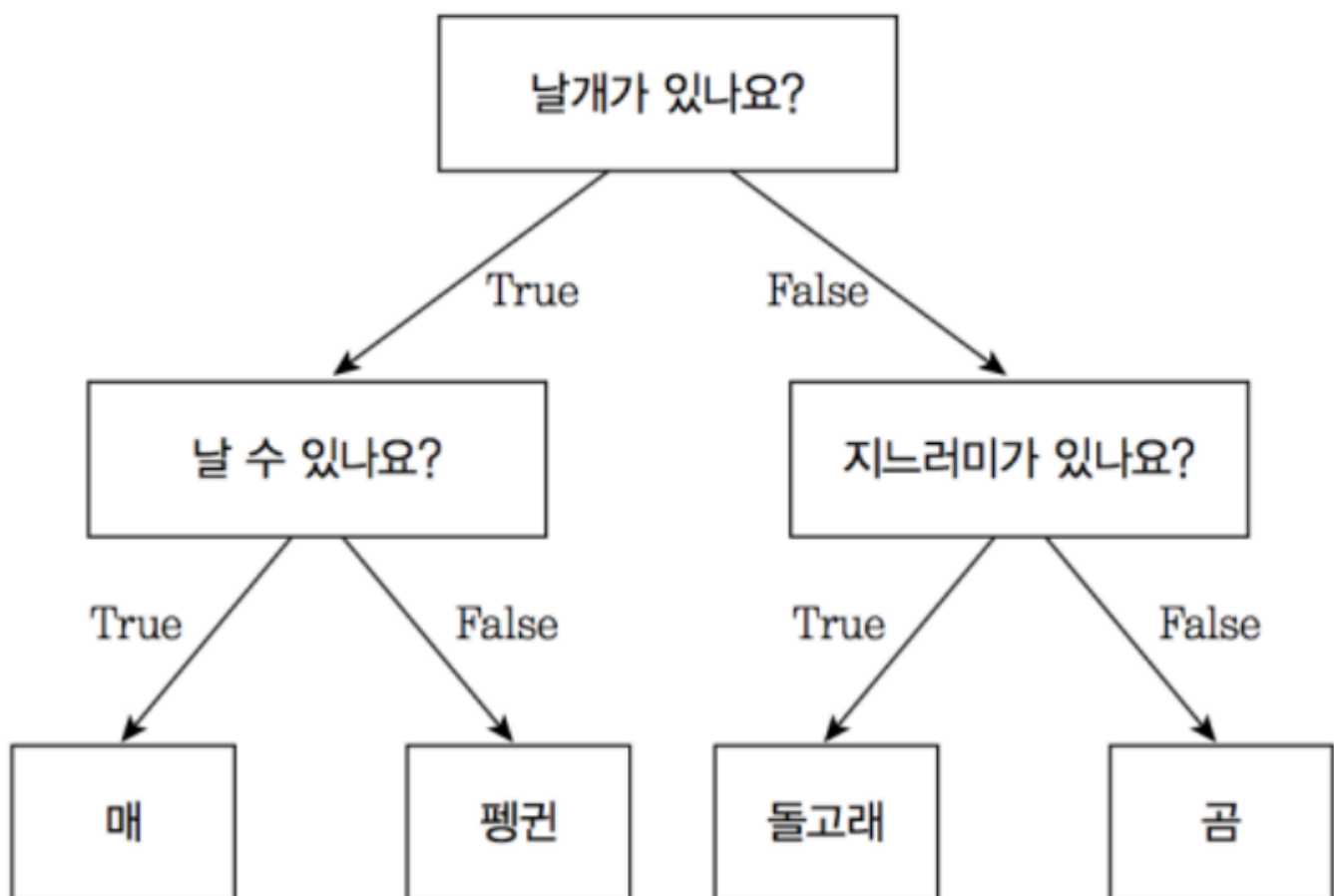
```

```
plt.legend()
plt.grid(True)
plt.show()

plot_svm_decision_boundary(svm_clf, X_scaled, y)
```

### 3. Decision Tree (결정 트리)

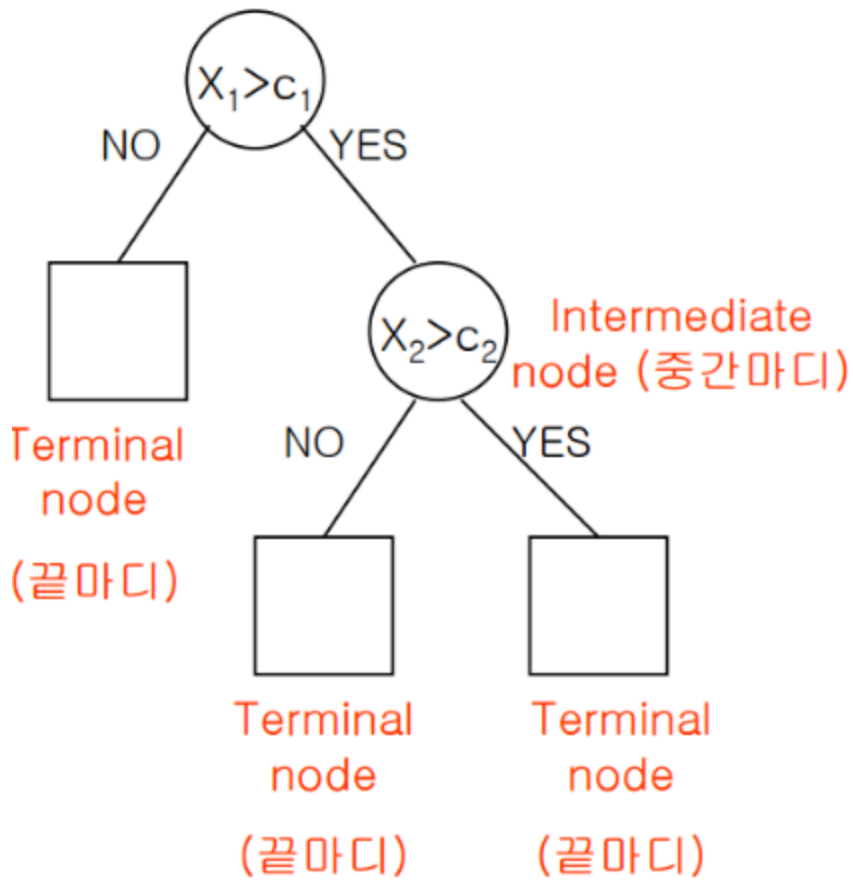
결정 트리 (Decision Tree, 의사 결정트리)는 분류(classification)와 회귀(Regression) 모두 가능한 지도 학습 모델 중 하나이다. 결정 트리는 스무고개 하듯이 예/아니오 질문을 이어가며, 학습한다.



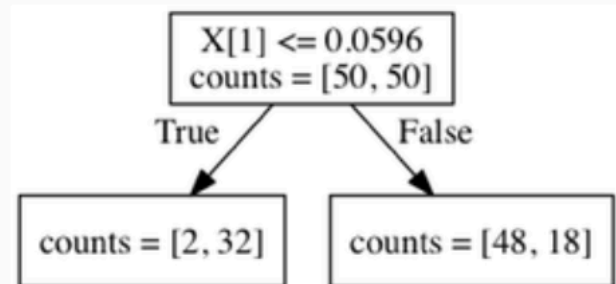
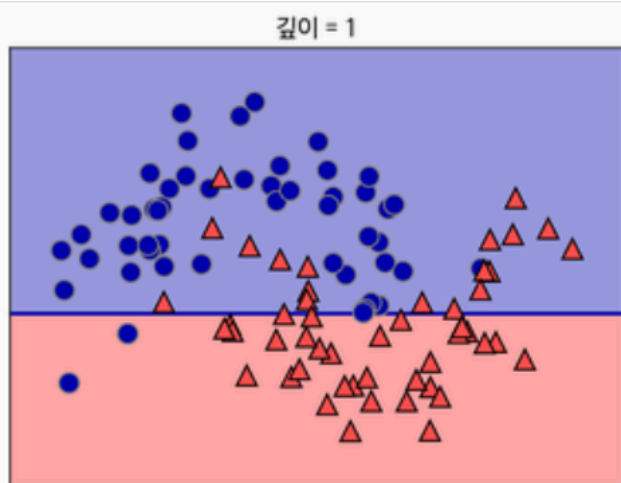
특정 기준(질문)에 따라 데이터를 구분하는 모델을 결정 트리 모델이라고 한다. 한 번의 분기때마다 변수 영역을 두 개로 구분한다. 결정 트리에서 질문이나 정답을 네모 상자를 (Node)라고 한다. 맨 처음 분류 기준 (즉, 첫 질문)을 Root Node라고 하고, 맨 마지막 노드를 Terminal Node 혹은 Leaf Node라고 한다.



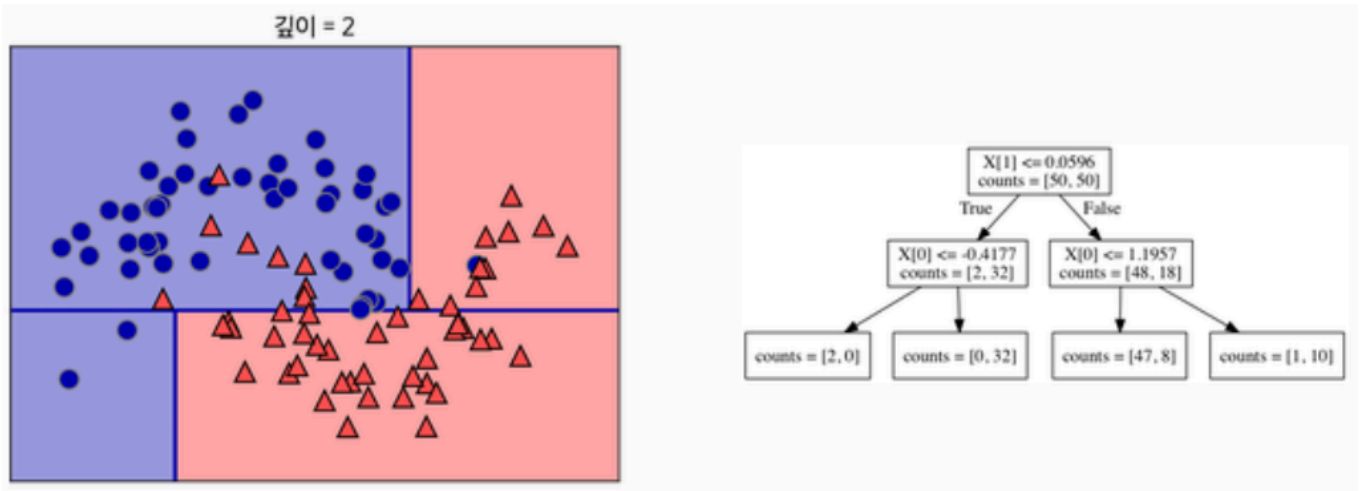
Root node (뿌리 마디)



Decion Tree 알고리즘 프로세스



데이터를 가장 잘 구분할 수 있는 질문을 기준으로 나눈다.



나눈 각 범주에서 또 다시 데이터를 가장 잘 구분할 수 있는 질문을 기준으로 나눈다. 이를 지나치게 많이 할 경우 과적합이 발생한다.

### 장점과 단점

- 장점
  - 결과 해석이 용이하며, 시각화에 적합.
  - 정규화나 스케일링 불필요.
  - 범주형 데이터도 잘 처리 가능.
- 단점
  - 과적합(overfitting)이 자주 발생.
  - 데이터에 민감하여 예측이 불안정할 수 있음
  - 클래스 불균형에 취약

### 실제 코드

```

from sklearn.datasets import make_moons
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. 데이터 준비
X, y = make_moons(n_samples=200, noise=0.25, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, random_state=42
)

# 2. 스케일링
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# 3. 모델 학습
clf = DecisionTreeClassifier(max_depth=3, random_state=42)
clf.fit(X_train_scaled, y_train)

# 4. 트리 구조 시각화
plt.figure(figsize=(12, 8))

plot_tree(
    clf,
    filled=True,
    feature_names=["Feature 1", "Feature 2"],
    class_names=["Class 0", "Class 1"]
)

plt.title("Decision Tree Structure")
plt.show()

```

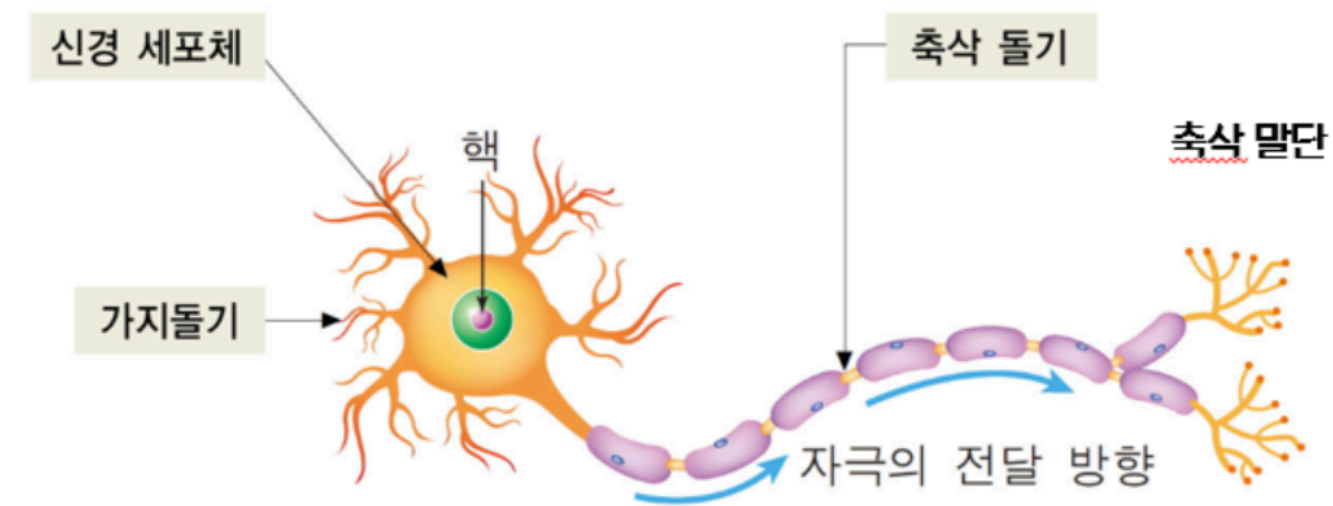
## 요약 비교

| 항목     | kNN               | SVM                | Decision Tree |
|--------|-------------------|--------------------|---------------|
| 학습 방식  | 메모리 기반(비훈련)       | 마진 최적화             | 재귀적 분할        |
| 예측, 속도 | 느림(거리 계산 필요)      | 빠름                 | 빠름            |
| 데이터 적합 | 소규모 데이터, 실시간 업데이트 | 고차원 데이터, 마진 문제에 적합 | 비선형/구조적 데이터   |

| 항목     | kNN           | SVM             | Decision Tree |
|--------|---------------|-----------------|---------------|
| 과적합 위험 | 중간            | 낮음              | 높음            |
| 장점     | 간단, 유연        | 일반화 우수, 마진 최적화  | 직관적, 시각화      |
| 단점     | 느린 예측, 이상치 민감 | 커널 선택 민감, 느린 학습 | 과적합, 불안정 구조   |

# 신경망 학습

## 생물학적 뉴런이란?



생물학적 뉴런은 사람이나 동물의 뇌를 구성하는 기본적인 정보 처리 단위이다. 뉴런은 외부 또는 다른 뉴런으로부터 신호를 받아들이고, 이를 처리한 후 다음 뉴런으로 전달하는 역할을 수행한다.

### 1. 기본 구조

생물학적 뉴런은 다음과 같은 주요 구성 요소를 가진다.

| 구성 요소                 | 기능                        |
|-----------------------|---------------------------|
| 수상 돌기(Dendrite)       | 다른 뉴런으로부터 신호를 수신          |
| 세포체 (Cell Body, Soma) | 수상 돌기에서 받은 신호를 통합 및 처리    |
| 축삭 (Axon)             | 처리된 신호를 다른 뉴런으로 전달        |
| 시냅스 (Synapse)         | 축삭 말단과 다음 뉴런 사이의 신호 전달 경로 |

## 2. 작동원리

1. 신호 수신: 다른 뉴런에서 시냅스를 통해 전달된 신호가 수상돌기로 유입됨
2. 신호 통합: 세포체에서 여러 입력 신호가 전기적 형태로 합산됨.
3. 임계값 도달: 입력의 총합이 일정 임계값을 넘으면, 액션 포텐셜이라 불리는 전기 신호가 생성된다.
4. 신호전파: 생성된 액션 포텐셜은 축삭을 따라 이동하며 다음 뉴런의 시냅스에 도달한다.

## 3. 신호의 강도

신호는 이진적(발화하거나 발화하지 않음)인 특성을 가지며, 시냅스의 강도는 학습이나 반복 경험에 의해 달라질 수 있음.

예를 들어, 자주 사용되는 시냅스는 강화되며 이는 장기 기억 형성에 기여한다.

## 4. 정보 처리 네트워크

뇌에는 수십억개의 뉴런이 존재하며 이들은 서로 복잡하게 연결되어 있다. 이러한 연결망은 감각 처리, 기억 사고, 감정 등의 복잡한 인지 활동을 가능하게 만든다.

# 인공 신경망과 퍼셉트론 개념 정리

## 인공신경망

인공신경망(Artificial Neural Network, ANN)은 생물학적 뉴런의 구조와 기능에서 영감을 받아 구성된 기계학습 모델이다. **인공지능은 인간의 학습, 추론, 지각 등의 지능적 능력을 컴퓨터가 수행할 수 있도록 만든 기술**이다. 이러한 기능을 구현하기 위해, 사람의 뇌 구조에서 영감을 받은 **인공신경망(Artificial Neural Network)** 이 도입되었으며, 그 기본 단위가 바로 **퍼셉트론(Perceptron)** 이다.

퍼셉트론은 생물학적 뉴런의 기능을 수학적으로 모방한 모델로, 입력을 받아 가중치를 적용하고 출력으로 전달하는 연산을 수행한다. 퍼셉트론은 딥러닝의 기초이자, 머신러닝에서 분류 문제를 해결하는 지도 학습 알고리즘으로 분류된다.

- 인공 신경망 구조 분류 및 퍼셉트론의 역할

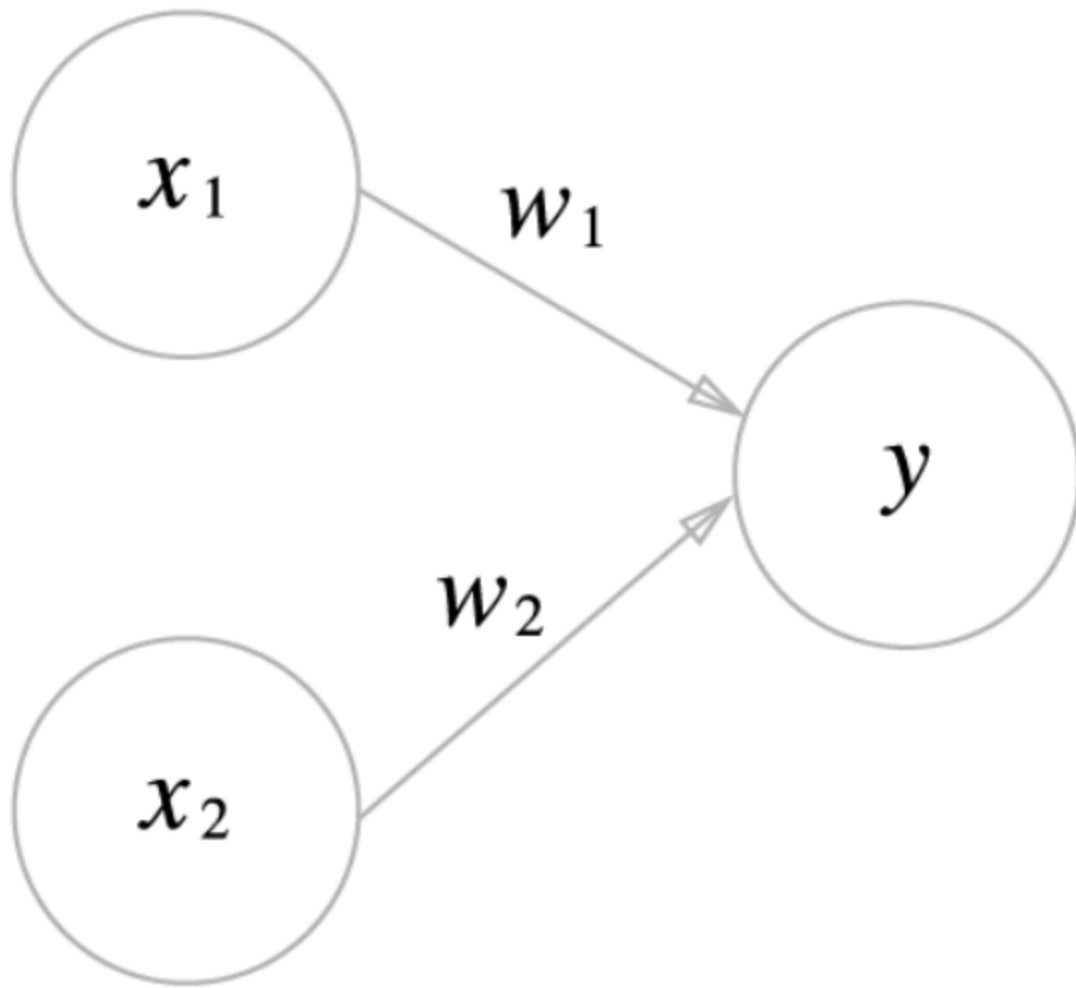
| 신경망 유형  | 구성 방식          | 퍼셉트론의 역할                |
|---------|----------------|-------------------------|
| 단층 퍼셉트론 | 입력층 → 출력층 (1층) | 단일 퍼셉트론으로 구성되어 선형 분류 수행 |

| 신경망 유형        | 구성 방식                | 퍼셉트론의 역할                   |
|---------------|----------------------|----------------------------|
| 다층 퍼셉트론 (MLP) | 입력층 → 은닉층 → 출력층      | 여러 퍼셉트론이 층을 이루며 복잡한 문제 처리  |
| 합성곱 신경망 (CNN) | 입력 → 합성곱층 → 풀링층 → 출력 | 이미지의 지역적 특징을 감지하는 연산 단위    |
| 순환 신경망 (RNN)  | 입력 → 은닉 상태 순환 → 출력   | 시간 순서 정보가 있는 데이터 처리에 사용된다. |

## 퍼셉트론의 정의

퍼셉트론은 인공 신경망의 **가장 기본적인 단위**로, 생물학적 뉴런의 동작을 모방한 이진 분류 모델이다. 1957년 프랭크 로젠블랫(Frank Rosenblatt)이 고안한 알고리즘이다. 입력값의 선형 조합을 통해 결과를 이진 분류한다. 단층 퍼셉트론은 오직 이진 분류 문제만 해결할 수 있으며, 다중 클래스 분류에는 직접 적용하기 어렵다.

퍼셉트론은  $n$  다수의 신호를 입력으로 받아 하나의 신호를 출력한다. 여기서 말하는 **신호**란 전류나 강물처럼 **흐름**이 있는 무언가를 상상하면 좋다. 전선을 따라, 전자를 흘려보내는 전류처럼, 퍼셉트론 신호도 **흐름을 만들고 정보를 앞으로 전달**한다. 다만 실제 전류와 달리 퍼셉트론 신호는 흐른다/안 흐른다(1 or 0)의 두가지 값을 가질 수 있다.



위의 그림은 입력공로 2개의 신호를 받은 퍼셉트론의 예이다.  $x_1$ 과  $x_2$ 는 입력 신호,  $y$ 는 출력 신호,  $w_1$ 과  $w_2$ 는 **가중치(weight)**를 뜻한다. 그림의 원은 **뉴런** 혹은 **노드**라고 부른다. 입력 신호가 뉴런에 보내질 때는 각각 고유한 가중치가 곱해진다.  $(w_1x_1, w_2x_2)$ , 뉴런에서 보내온 신호의 총합이 정해진 한계를 넘어설 때만을 1을 출력한다. 이러한 한계를 **임계값**이라고 하며,  $\theta$ (세타) 기호로 나타낸다.

2개 입력에 대한 퍼셉트론은 다음과 같은 수식으로 나타낼 수 있다.

$$y = \begin{cases} 0, & \text{if } w_1x_1 + w_2x_2 \leq \beta \\ 1, & \text{if } w_1x_1 + w_2x_2 > \beta \end{cases}$$

퍼셉트론은 복수의 입력 신호에 각각에 고유한 가중치를 부여하고 가중치는 각 신호가 결과에 주는 영향력을 조절하는 요소로 작용한다. 즉, 가중치가 클수록 해당 신호가 그 만큼 더 중요함을 뜻한다.

## 논리 회로 구현

퍼셉트론을 활용해 논리회로를 구현할 수 있다.

## AND 게이트

| $x_1$ | $x_2$ | $y$ |
|-------|-------|-----|
| 0     | 0     | 0   |
| 1     | 0     | 0   |
| 0     | 1     | 0   |
| 1     | 1     | 1   |

해당 논리회로를 파이썬으로 구현한다.  $x_1$ 과  $x_2$ 를 인수로 받는 AND 함수이다.

PYTHON

```
def AND(x1, x2):
    w1, w2, theta = 0.5, 0.5, 0.7
    tmp = x1 * w1 + x2 * w2

    if tmp <= theta:
        return 0
    elif tmp > theta:
        return 1

print(AND(0, 0)) # 0
print(AND(1, 0)) # 0
print(AND(0, 1)) # 0
print(AND(1, 1)) # 1
```

- 매개변수 **w1, w2, theta**는 함수안에서 초기화하고, 가중치를 곱한 입력의 총합이 임계값을 넘으면 1을 반환하고 그 외에는 0을 반환한다.

### 가중치와 편향 도입

$$y = \begin{cases} 0, & \text{if } w_1x_1 + w_2x_2 + b \leq 0 \\ 1, & \text{if } w_1x_1 + w_2x_2 + b > 0 \end{cases}$$

여기서 **b**를 편향이라고 한다.  $w_1, w_2$ 는 그대로 가중치이다.

퍼셉트론은 입력 신호에 가중치를 곱한 값과 편향을 합하여, 그 값이 0을 넘으면 1을 출력하고 그렇지 않으면 0을 출력한다.



```
import numpy as np

x = np.array([0, 1])
w = np.array([0.5, 0.5])
b = -0.7

print(np.sum(w*x)+b)
```

## 가중치

**가중치(weight)**는 각 입력 신호가 결과에 주는 **영향력(중요도)**을 조정하는 매개변수이다.

- 예를 들어 가중치가 크면 입력값이 더 커져 훨씬 쉽게 임계값을 넘기기 쉬워지고 가중치가 작으면 입력값과 곱해져 임계값을 넘기가 어려워진다.

## 편향

**편향(bias)** 편향이 얼마나 쉽게 활성화(결과로 1을 출력)하느냐를 조정하는 매개변수이다.

- 예를 들어  $b = 0.1$ 이면 각 입력 신호에 가중치를 곱한 값들의 합이 0.1을 초과할 때만 뉴런이 활성화한다. 반면  $b = -20$  각 입력 신호에 가중치를 곱한 값들의 합이 20.0을 넘지 않으면 뉴런이 활성화하지 않는다.

## 가중치와 편향 구현하기

```
import numpy as np

def AND(x1, x2):
    x = np.array([0, 1])
    w = np.array([0.5, 0.5])
    b = -0.7

    tmp = np.sum(w*x)+b

    if tmp <= 0:
        return 0
    elif tmp > 1:
        return 1
```

## NAND 게이트와 OR 게이트

- NAND게이트와 OR 게이트도 신경망으로 구현이 가능하다.

## NAND

| $x_1$ | $x_2$ | $y$ |
|-------|-------|-----|
| 0     | 0     | 1   |
| 1     | 0     | 1   |
| 0     | 1     | 1   |
| 1     | 1     | 0   |

코드는 아래처럼 구현가능하다

PYTHON

```
import numpy as np

def NAND(x1, x2):
    x = np.array([x1, x2])
    w = np.array([-0.5, -0.5])
    b = 0.7

    tmp = np.sum(w * x) + b

    if tmp <= 0:
        return 0
    else:
        return 1

print(NAND(0, 0)) # 1
print(NAND(1, 0)) # 1
print(NAND(0, 1)) # 1
print(NAND(1, 1)) # 0
```

## OR

| $x_1$ | $x_2$ | $y$ |
|-------|-------|-----|
| 0     | 0     | 0   |
| 1     | 0     | 1   |
| 0     | 1     | 1   |
| 1     | 1     | 1   |

아래 코드처럼 구현할 수 있다.

```
import numpy as np

def NAND(x1, x2):
    x = np.array([x1, x2])
    w = np.array([0.5, 0.5])
    b = -0.2

    tmp = np.sum(w * x) + b

    if tmp <= 0:
        return 0
    else:
        return 1

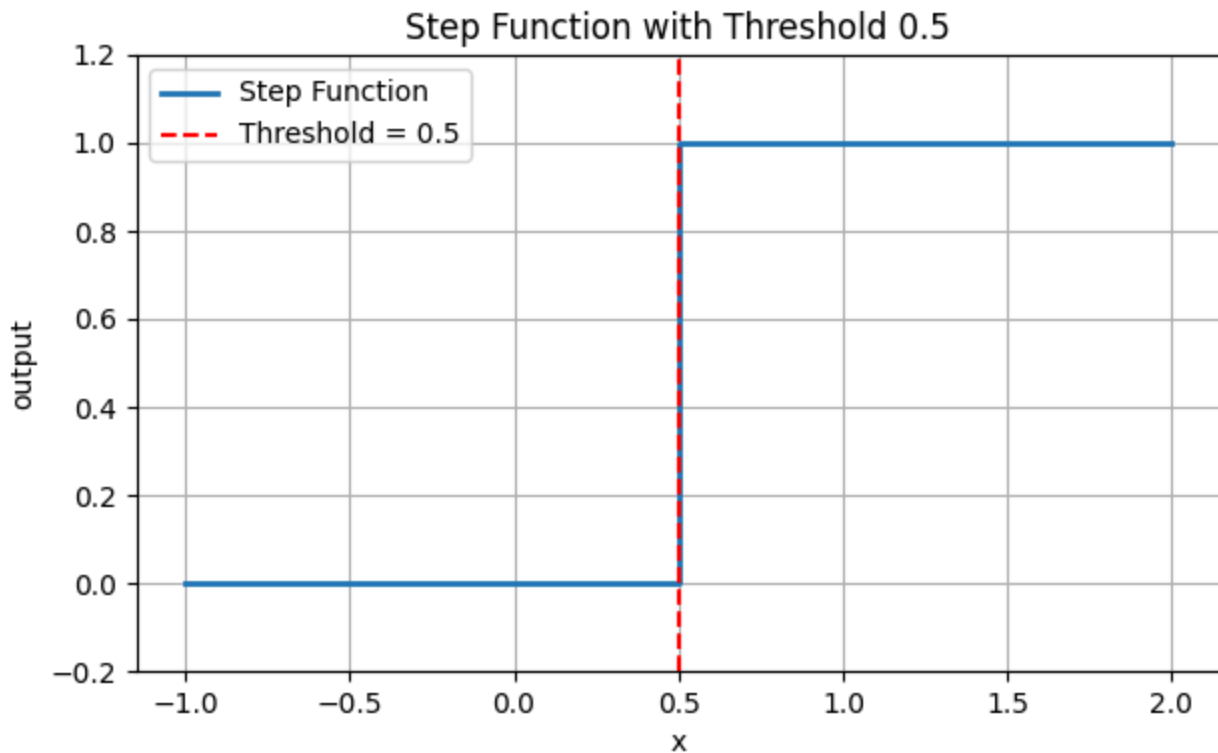
print(NAND(0, 0)) # 1
print(NAND(1, 0)) # 1
print(NAND(0, 1)) # 1
print(NAND(1, 1)) # 0
```

## 단층 퍼셉트론의 구조 및 수식

| 구성요소                                     | 설명                        |
|------------------------------------------|---------------------------|
| 입력값( $x_1, x_2, \dots, x_n$ )            | 각 입력 변수 (예: 픽셀 값, 특징 값 등) |
| 가중치( $w_1, w_2, \dots, w_n$ )            | 입력의 중요도를 조절하는 계수          |
| 편향( $b$ )                                | 결정 경계를 이동시키는 값            |
| 총합 ( $z = w_1x_1 + w_2x_2 + \dots + b$ ) | 가중 합                      |

$$\text{output} = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

- $w_i$  : 가중치
- $x_i$  : 입력값
- $b$  : 편향
- $f$  : 활성화 함수



계단 함수를 통해 좀더 설명해볼 수 있다. 계단 함수는  $x$ 값이 임계값(0.5) 이상이면 1을 출력하고  $x$  값이 0값이하면 0을 출력하는 활성화 함수이다. 이렇게 활성화 함수를 거쳐 출력값은 1이 된다. 여기서 활성화 함수가  $f$ 라고 할 수 있다.

## 단층 퍼셉트론의 학습 방법

퍼셉트론은 **오차 기반 학습법**을 통해 가중치를 수정한다.

가중치 업데이트 식:

$$w_i \leftarrow w_i + \eta \cdot (y - \hat{y}) \cdot x_i$$

편향 업데이트 식:

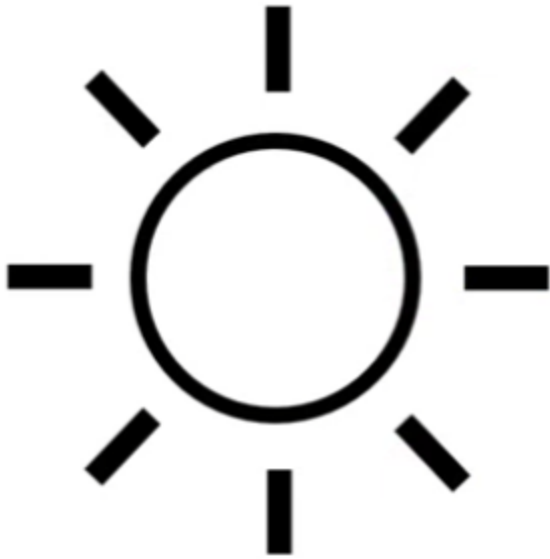
$$b \leftarrow b + \eta \cdot (y - \hat{y})$$

- $w_i$  :  $i$ 번째 가중치
- $b$  : 편향
- $\eta$  : 학습률
- $y$  : 실제 정답
- $\hat{y}$  : 모델의 예측값
- $x_i$  :  $i$ 번째 입력값

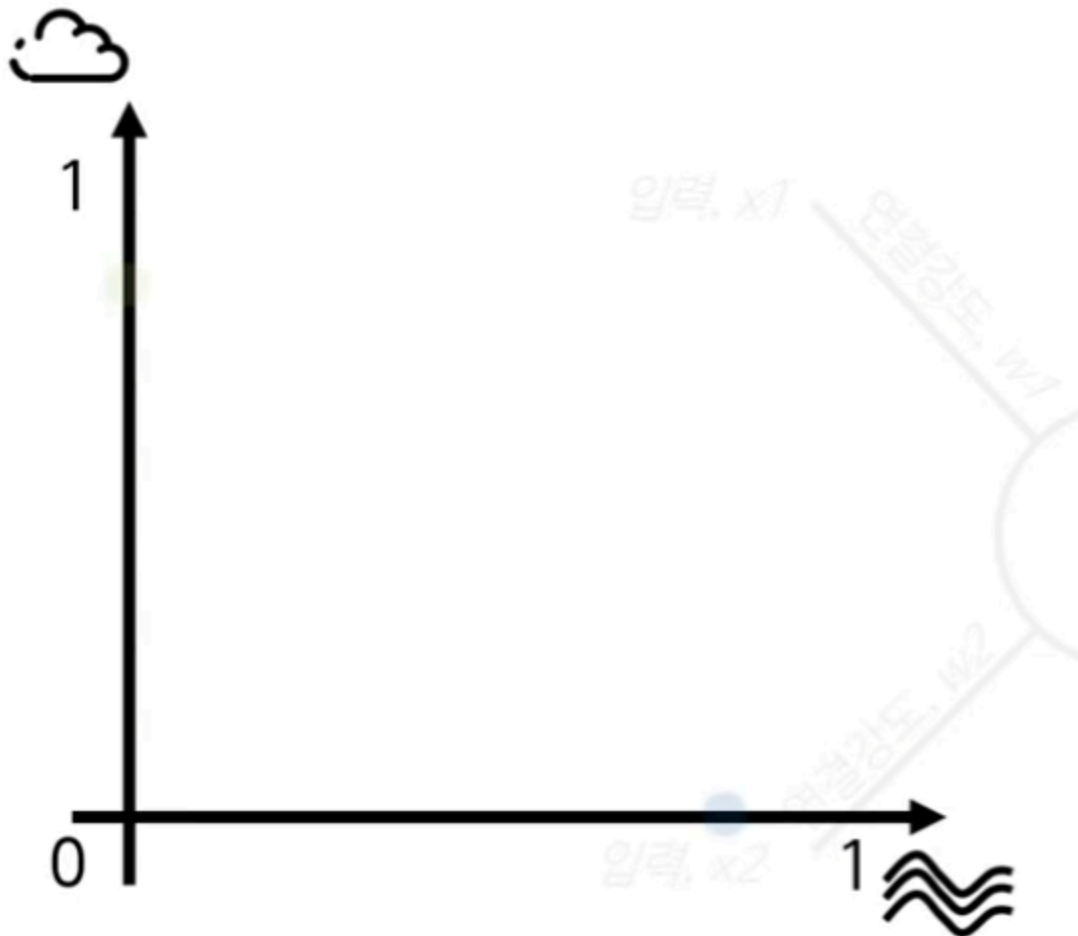
## 실제 예시

지금까지의 교재 내용만으로 AND나 OR 문제들은 구현할 수 있다는 것은 이해할 수 있을것이다. 다만 이 내용만으로 실제로 어떻게 적용할 수 있을지 예시를 통해 이해해 보겠다.

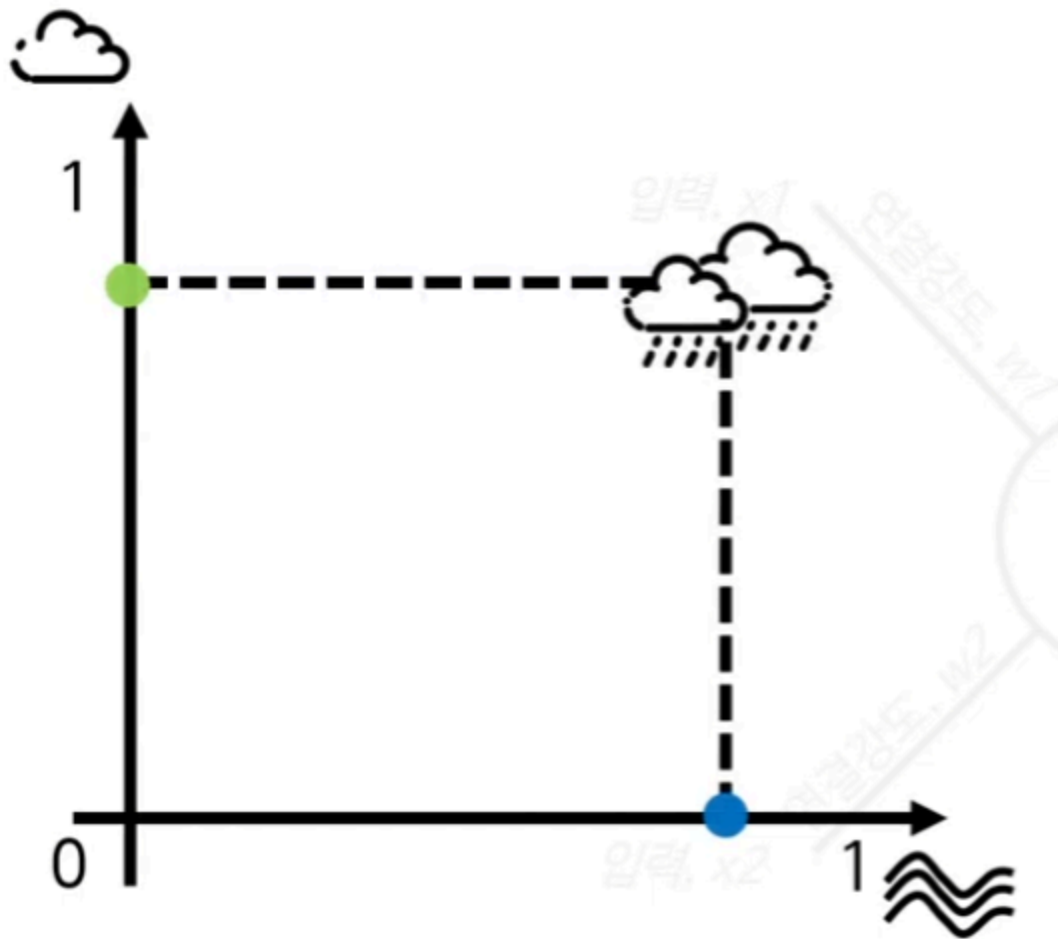
먼저 날씨를 예측하는 퍼셉트론을 구현해 본다고 해보자



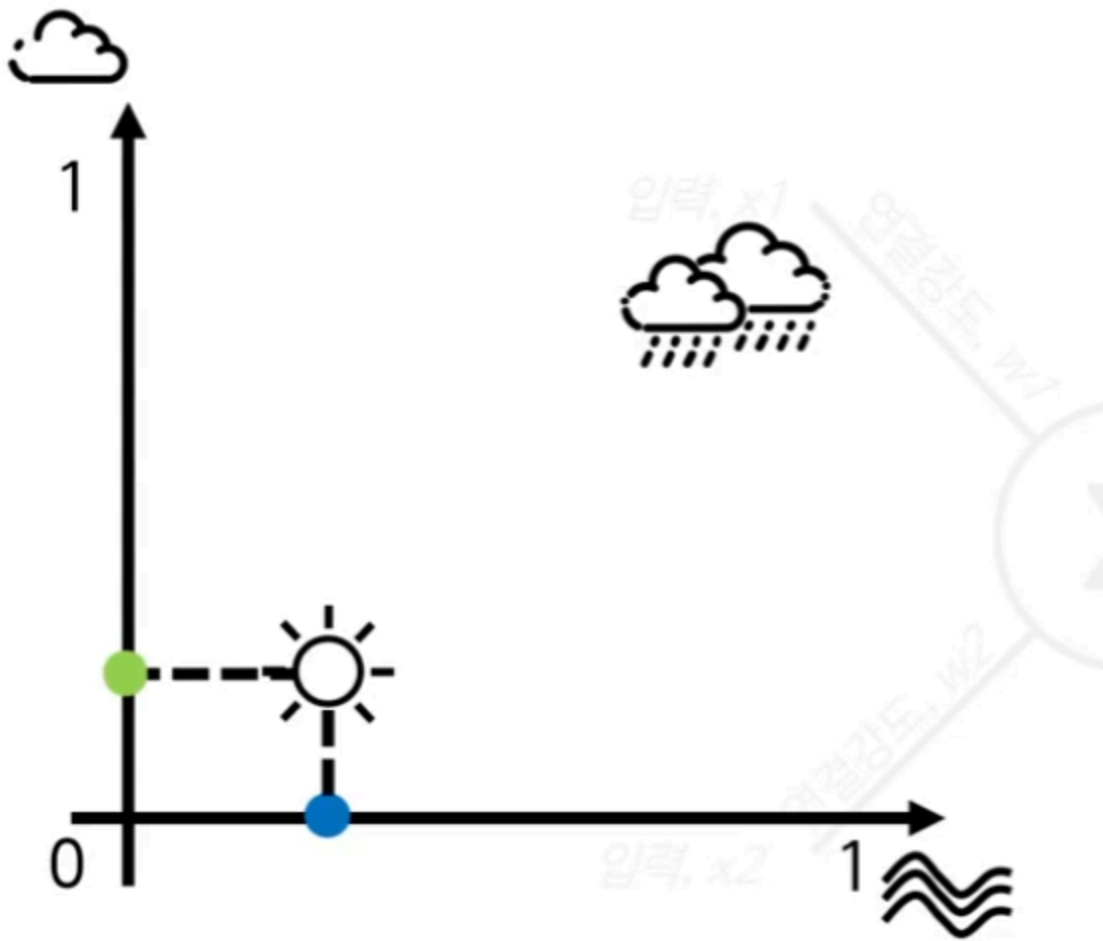
예를 들어 날씨를 예측하는 조건을 구름과 바람으로 가정해본다.



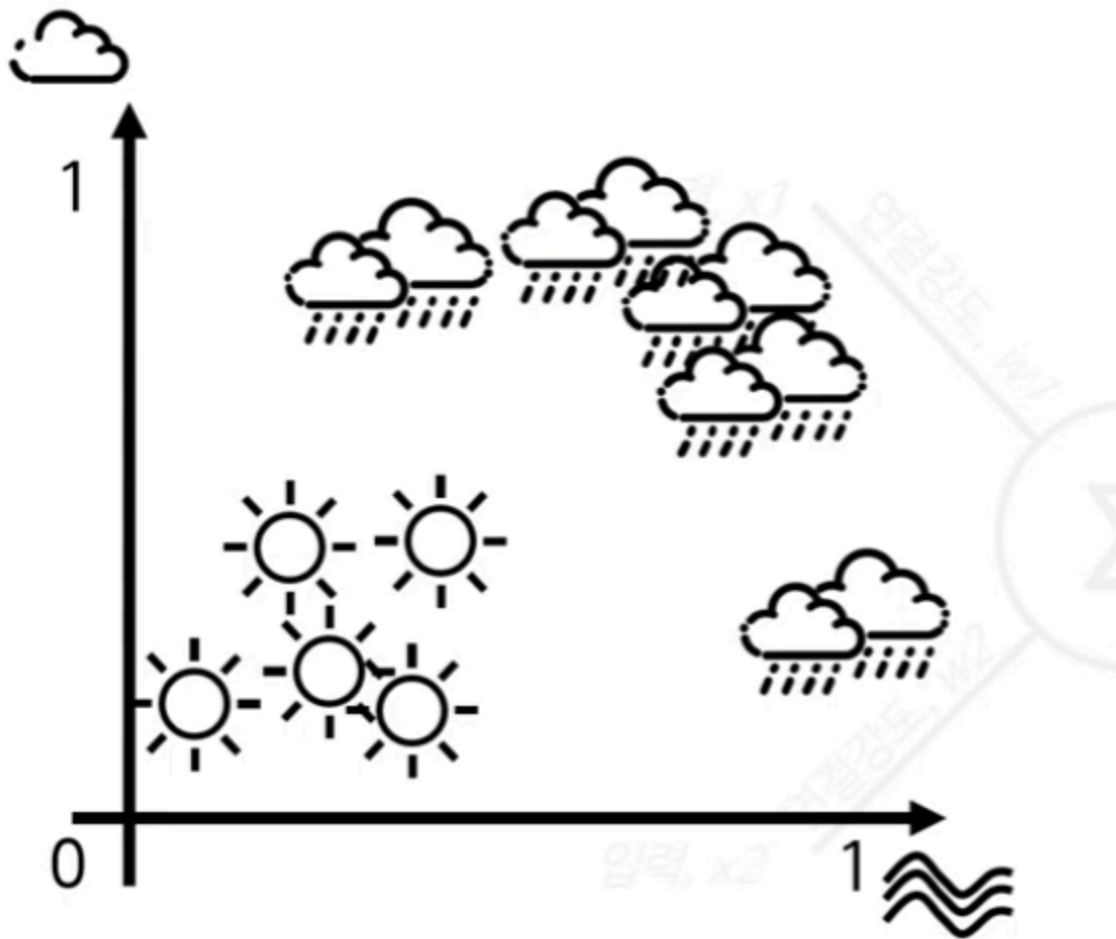
바람의 많고 적음을 0과 1사이로 표현할 수 있고 바람의 많고 적음도 0과 1사이로 표현할 수 있다.



구름도 많고 바람도 많으면 비가올 확률이 높고



구름이 적고 바람도 적으면 해 가 뜰 확률이 높다.



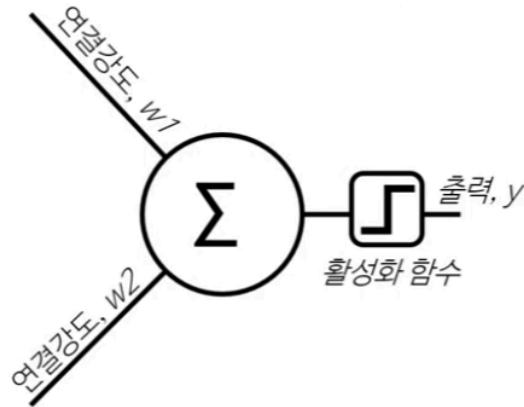
실제로 이렇게 날씨가 맑은 경우와 비가 온 경우를 모아서 수치로 정리한 것을 데이터 셋이라고 한다.

| 구름의 양 | 바람의 양 | 날씨 |
|-------|-------|----|
| 0.8   | 0.6   | 비  |
| 0.6   | 0.9   | 비  |
| 0.1   | 0.2   | 해  |
| 0.3   | 0.1   | 해  |
| 0.6   | 0.6   | 비  |
| 0.4   | 0.3   | 해  |
| 0.1   | 0.2   | 해  |

이 데이터를 퍼셉트론에 넣어 본다.



| x1  | x2  |
|-----|-----|
| 0.8 | 0.6 |
| 0.6 | 0.9 |
| 0.1 | 0.2 |
| 0.3 | 0.1 |
| 0.6 | 0.6 |
| 0.4 | 0.3 |
| 0.1 | 0.2 |



| y |
|---|
| 1 |
| 1 |
| 0 |
| 0 |
| 1 |
| 0 |
| 0 |

여기서  $y$ 는 실제 라벨 값이다.

이 값을  $\text{output} = f(\sum_{i=1}^n w_i x_i + b)$ 에 대입해보면 아래와 같은 표로 나온다.

- 여기서  $b=0.5$ 이다.

| 구름의 양 $x_1$ | 바람의 양 $x_2$ | 실제 값 | 계산 과정                             | 예측값  |
|-------------|-------------|------|-----------------------------------|------|
| 0.8         | 0.6         | 1    | $0.8 \times 0.8 + 0.6 \times 0.6$ | 1.00 |
| 0.6         | 0.9         | 1    | $0.8 \times 0.6 + 0.6 \times 0.9$ | 1.02 |
| 0.1         | 0.2         | 0    | $0.8 \times 0.1 + 0.6 \times 0.2$ | 0.20 |
| 0.3         | 0.1         | 1    | $0.8 \times 0.3 + 0.6 \times 0.1$ | 0.30 |
| 0.6         | 0.6         | 1    | $0.8 \times 0.6 + 0.6 \times 0.6$ | 0.84 |
| 0.4         | 0.3         | 0    | $0.8 \times 0.4 + 0.6 \times 0.3$ | 0.50 |
| 0.1         | 0.2         | 0    | $0.8 \times 0.1 + 0.6 \times 0.2$ | 0.20 |

이처럼 퍼셉트론은 입력값과 가중치를 이용해 날씨를 예측할 수 있다.

현재 예시는 구름의 양과 바람의 양, 두 가지 조건만 사용하고 있기 때문에 매우 단순한 모델이다.

하지만 실제 날씨는 구름과 바람만으로 결정되지 않는다.

습도, 기온, 기압, 기압 변화량, 풍속, 풍향, 이전 날씨, 일사량, 계절과 같은 다양한 조건들이 함께 영향을 준다.

따라서 입력값을 더 많이 추가하면 모델은 날씨를 판단할 때 더 많은 정보를 사용할 수 있다.

예를 들어 구름의 양이 많더라도 습도가 낮고 기압이 높으며 일사량이 강하다면 실제로는 비가 오지 않을 수도 있다.

반대로 구름의 양이 아주 많지 않더라도 습도가 높고 기압이 빠르게 낮아지며 이전 날씨가 흐렸다면 비가 올 가능성이 높아질 수 있다.

| 추가 조건  | 변수 예시    |
|--------|----------|
| 습도     | $x_3$    |
| 기온     | $x_4$    |
| 기압     | $x_5$    |
| 기압 변화량 | $x_6$    |
| 풍속     | $x_7$    |
| 풍향     | $x_8$    |
| 이전 날씨  | $x_9$    |
| 강수 확률  | $x_{10}$ |
| 일사량    | $x_{11}$ |
| 계절     | $x_{12}$ |

즉, 입력값을 늘린다는 것은 모델이 판단에 사용할 수 있는 정보를 늘리는 것과 같다.

관련 있는 조건들이 충분히 추가되고, 그에 맞는 데이터로 학습이 이루어진다면 단순한 예제 수준을 넘어 실제 날씨 예측에 더 가까운 모델을 만들 수 있다.

기존 모델은 두 개의 입력값만 사용했다.

$$score = w_1x_1 + w_2x_2$$

하지만 조건을 더 추가하면 다음과 같이 확장할 수 있다.

$$score = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + \cdots + w_{12}x_{12}$$

여기서 각 입력값은 날씨에 영향을 주는 조건이고, 각 가중치는 해당 조건이 예측 결과에 얼마나 큰 영향을 주는지를 의미한다.

## 학습 예시

예를 들어 습도가 비 예측에 큰 영향을 준다면  $w_3$ 의 값이 크게 조정될 수 있고, 일사량이 강할수록 비가 올 가능성이 낮아진다면  $w_{11}$ 은 음수 방향의 가중치를 가질 수도 있다.

따라서 학습이란 실제 날씨와 예측 결과의 차이를 줄이도록 각 조건의 가중치를 조절하는 과정이라고 볼 수 있다.

여기서 실제값과 예측값의 차이를 최소화 하도록 퍼셉트론의 가중치를 조절하는 것을 학습 (learning) 이라고 한다. 즉 퍼셉트론 알고리즘은 다음과 같다. 구름의 양 =  $x_1$ , 바람의 양 =  $x_2$ , 습도 =  $x_3$ , 온도 =  $x_4$ ,  $x_5$

| 구름의 양 $x_1$ | 바람의 양 $x_2$ | 실제 값 | 계산 과정                             | 예측값  |
|-------------|-------------|------|-----------------------------------|------|
| 0.8         | 0.6         | 1    | $0.8 \times 0.8 + 0.6 \times 0.6$ | 1.00 |
| 0.6         | 0.9         | 1    | $0.8 \times 0.6 + 0.6 \times 0.9$ | 1.02 |
| 0.1         | 0.2         | 0    | $0.8 \times 0.1 + 0.6 \times 0.2$ | 0.20 |
| 0.3         | 0.1         | 1    | $0.8 \times 0.3 + 0.6 \times 0.1$ | 0.30 |
| 0.6         | 0.6         | 1    | $0.8 \times 0.6 + 0.6 \times 0.6$ | 0.84 |
| 0.4         | 0.3         | 0    | $0.8 \times 0.4 + 0.6 \times 0.3$ | 0.50 |
| 0.1         | 0.2         | 0    | $0.8 \times 0.1 + 0.6 \times 0.2$ | 0.20 |

새로운 가중치 = 현재 가중치 + 현재 입력값  $\times$  오차  $\times$  학습률

여기서 오차는

오차 = 실제값 - 예측값

이라고 표현할 수 있다. 따라서 오차값이 0이면 나머지 **현재 입력값  $\times$  오차  $\times$  학습률** 이 0이 되므로 가중치는 변하지 않는다.

하지만 오차가 0이 아니라면 가중치가 변경된다. 현 입력 값이 클수록 오차의 발생에 기여한 바가 크다고 보고, 새로운 가중치를 업데이트 할 때 변화를 더 많이 주는 방향으로 학습이 이루어진다.

따라서 현 입력값에 오차를 곱하는 의미는 큰 입력값이 뉴런의 발화를 일으키고 그로 인해 오차의 발생에 기여한 바가 크다고 보고, 그 기여한 정도에 따라 연결 강도를 더하여 오차를 줄이는 새로운 가중치를 아래의 표처럼 구할 수 있다.

| 순서 | 구름<br>$x_1$ | 바람<br>$x_2$ | 실제<br>날씨 | 실제<br>값 $y$ | 현재<br>$w_1$ | 현재<br>$w_2$ | 계산 과정                             | score |
|----|-------------|-------------|----------|-------------|-------------|-------------|-----------------------------------|-------|
| 1  | 0.8         | 0.6         | 비        | 1           | 0.8         | 0.6         | $0.8 \times 0.8 + 0.6 \times 0.6$ | 1.00  |
| 2  | 0.6         | 0.9         | 비        | 1           | 0.8         | 0.6         | $0.8 \times 0.6 + 0.6 \times 0.9$ | 1.02  |
| 3  | 0.1         | 0.2         | 해        | 0           | 0.8         | 0.6         | $0.8 \times 0.1 + 0.6 \times 0.2$ | 0.20  |
| 4  | 0.3         | 0.1         | 해        | 0           | 0.8         | 0.6         | $0.8 \times 0.3 + 0.6 \times 0.1$ | 0.30  |
| 5  | 0.6         | 0.6         | 비        | 1           | 0.8         | 0.6         | $0.8 \times 0.6 + 0.6 \times 0.6$ | 0.84  |
| 6  | 0.4         | 0.3         | 해        | 0           | 0.8         | 0.6         | $0.8 \times 0.4 + 0.6 \times 0.3$ | 0.50  |
| 7  | 0.1         | 0.2         | 해        | 0           | 0.8         | 0.6         | $0.8 \times 0.1 + 0.6 \times 0.2$ | 0.20  |

## XOR 문제와 단층 퍼셉트론의 한계

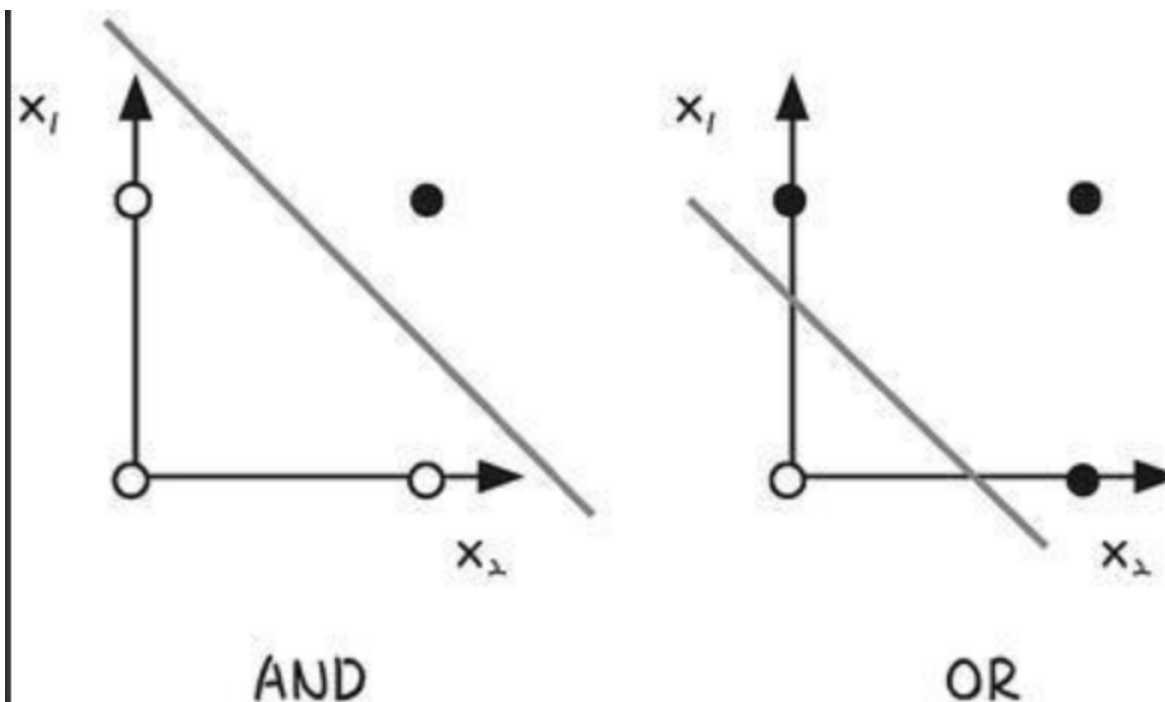
### XOR 문제

XOR 문제 베타적 논리합 문제이다. 아래 표와 같이  $x_1$ 과  $x_2$  중 한쪽이 1일 때만 1을 출력한다.

| $x_1$ | $x_2$ | $y$ |
|-------|-------|-----|
| 0     | 0     | 1   |
| 1     | 0     | 0   |
| 0     | 1     | 0   |
| 1     | 1     | 1   |

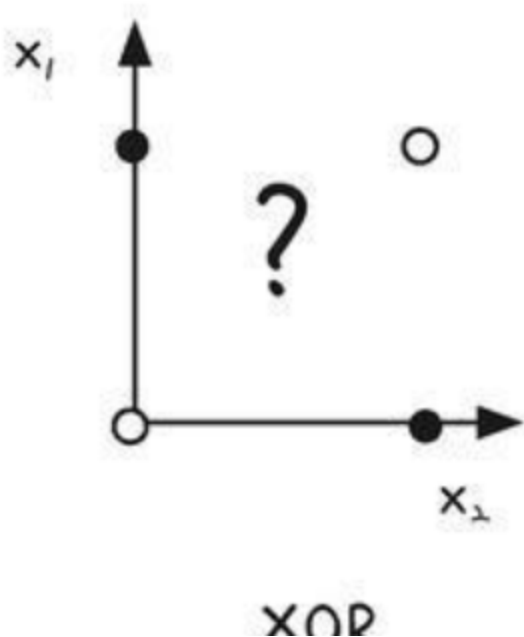
### 단층 퍼셉트론의 한계

지금까지의 퍼셉트론으로는 XOR 게이트를 구현할 수 없다. 그 이유는 XOR 문제는 비선형 분류 문제이기 때문이다.



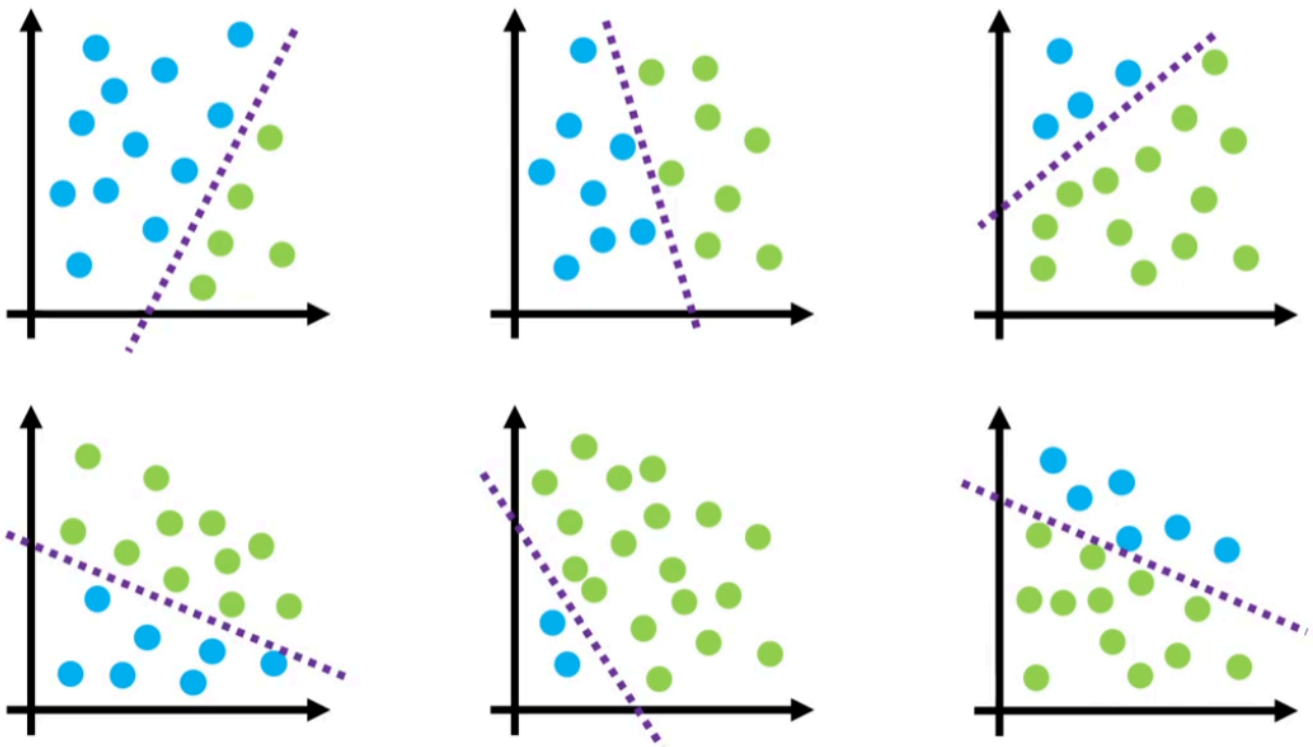
위의 그림은 (0, 0), (0, 1), (1, 0), (1, 1)의 상황에서 결과값이 1이면 흰 동그라미, 결과값이 0이면 검은색 동그라미를 그려 AND와 OR을 표현했다.

AND와 OR 문제를 풀기 위해서는 흰 동그라미와 검은색 동그라미가 섞이지 않도록 영역을 나눠야 한다. 실제로 그림의 네 점은 제대로 나누고 있다.

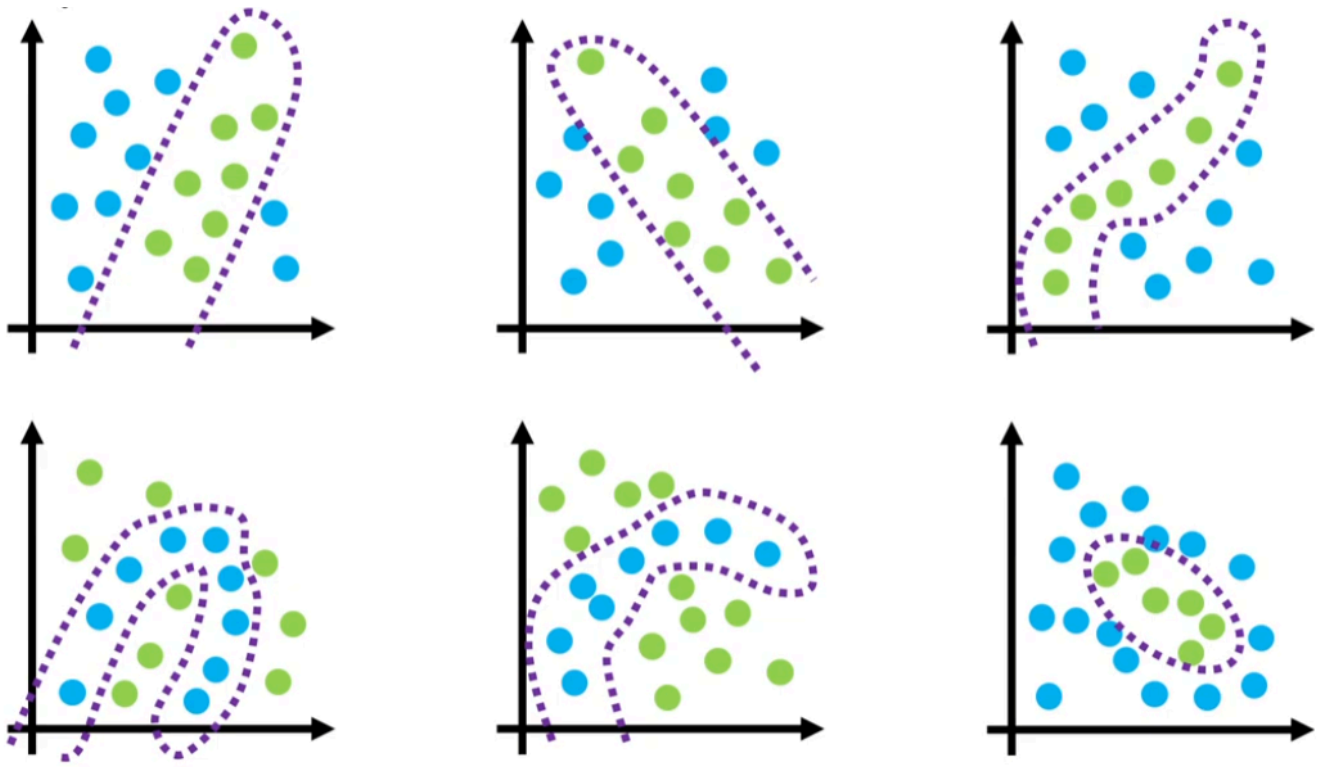


위 그림처럼 XOR의 경우 AND 문제나, OR 문제 처럼 직선 하나로 나누는 영역을 나눌 수 없다.

즉 퍼셉트론은 선형 분리가능한 데이터들을 분류해내는 하나의 **선형 분류기(Linear Classifier)** 라고 볼 수 있다.



선 하나로 분류할 수 있는 데이터셋이면 퍼셉트론은 학습(Learning)을 통해 가중치 값을 조절하여 어떠한 데이터셋이라도 잘 분류해낼 수 있다.



위 그림처럼 선형 분리가 불가능한 데이터셋은 퍼셉트론은 학습할 수 없다. **퍼셉트론이 XOR 문제를 풀지 못하는 이유도 선형 분리가 불가능한 문제이기 때문이다.**

이러한 한계를 극복하기 위해 **다층 퍼셉트론 (MLP)**, 즉 은닉층이 있는 신경망이 등장했다.

```

# 라이브러리 불러오기
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

# 1. Create synthetic data
X, y = make_moons(n_samples=300, noise=0.3, random_state=0)

# 2. Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)

# 3. Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
X_all_scaled = scaler.transform(X)

# 4. Meshgrid for plotting
def plot_decision_boundary(knn, X, y, ax, k):
    h = 0.02

    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

    xx, yy = np.meshgrid(
        np.arange(x_min, x_max, h),
        np.arange(y_min, y_max, h)
    )

    Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    cmap_light = ListedColormap(['#FFBBBB', '#BBFFBB'])
    cmap_bold = ListedColormap(['#FF0000', '#00AA00'])

    ax.contourf(xx, yy, Z, cmap=cmap_light, alpha=0.6)
    ax.scatter(X[:, 0], X[:, 1], c=y, cmap=cmap_bold, edgecolor='k')

```

```
ax.set_title(f"k = {k}")
ax.set_xlim(xx.min(), xx.max())
ax.set_ylim(yy.min(), yy.max())

# 5. Try different k values
k_values = [1, 3, 5, 15]

fig, axes = plt.subplots(1, len(k_values), figsize=(18, 4))

for ax, k in zip(axes, k_values):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)

    plot_decision_boundary(knn, X_all_scaled, y, ax, k)

plt.tight_layout()
plt.show()
```